

Name: B. Ramya

Capstone Project

DineDash Food Delivery Data Analysis

BACKGROUND:

The food delivery industry has experienced rapid growth over the last decade, driven by the increasing demand for convenience, improved delivery logistics, and the expansion of digital platforms. MealDash has revolutionized the way food is ordered and delivered, creating new opportunities for restaurant owners and consumers alike.

However, as the industry continues to expand, food delivery platforms face increasing competition, rising customer expectations for faster and more personalized services, and challenges in optimizing their operations across a diverse and extensive menu offering.

In this case study, we focus on the use of big data and advanced analytics to streamline operations, optimize delivery times, enhance customer experience, and improve menu offerings. By leveraging data processing platforms such as Apache Spark and Databricks, we can better understand customer preferences, predict demand, and offer personalized recommendations, ultimately driving more efficient and profitable business operations.

CHALLENGES:

DineDash has to address several challenges to maintain its competitive advantage and operational efficiency:

1. Data Fragmentation:

Different systems for orders, customer profiles, menus, and delivery logs lead to fragmented data, making it difficult to analyze information comprehensively and derive meaningful insights.

2. Customer Segmentation:

Understanding customer preferences based on historical data is essential for offering personalized promotions, menu recommendations, and loyalty rewards.

3. Order Prediction:

Predicting order volumes across different regions and times of the day helps optimize delivery capacity, allocate resources effectively, and reduce customer wait times.

4. Menu Optimization:

Identifying the most popular and profitable menu items helps improve menu offerings. DineDash also needs to identify suitable item combinations for customers and ensure that menu availability is updated in real time.

5. Order Fulfillment Delays:

Ensuring on-time deliveries and improving operational workflows can be challenging due to external factors such as weather conditions, traffic congestion, and restaurant preparation delays.

6. Promotions and Discounts:

Identifying the most effective promotional offers and discounts is important for increasing order value and encouraging repeat purchases while maintaining overall profitability.

BUSINESS NEEDS:

To stay competitive and meet customer demands, DineDash needs to address the following business needs:

1. Data Integration:

Integrating data from different sources, including customer orders, restaurant data, delivery times, and other operational information, to build a comprehensive view of customer behaviour, restaurant performance, and delivery operations.

2. Real-Time Analytics:

Implementing a real-time data processing pipeline that analyzes orders as they come in, providing instant insights and enabling dynamic decision-making, such as dynamic pricing and offer recommendations.

3. Predictive Analytics:

Using machine learning techniques to predict demand, customer preferences, delivery times, and possible bottlenecks in the system.

4. Optimization Algorithms:

Implementing algorithms that optimize the delivery process by calculating the best routes, identifying available delivery agents, and reducing overall food delivery time.

5. Personalization:

Offering personalized promotions, product recommendations, and tailored menus based on customers' past behaviour, location, and preferences.

6. Performance Tracking:

Continuously monitoring key performance indicators (KPIs), such as customer satisfaction, order fulfilment rate, delivery time, and revenue per order.

PROPOSED SOLUTION:

The proposed solution involves setting up a big data processing pipeline using Databricks and Apache Spark, along with machine learning models to provide real-time insights and predictive analytics. The solution leverages the following components:

1. Data Lake:

A centralized repository for storing all structured and unstructured data, including order details, customer information, menu data, and delivery logs.

2. ETL Pipelines:

Using Delta Live Tables (DLT) to manage the transformation of raw data into usable formats for analysis. The pipeline will clean, enrich, and structure the data for downstream analytics.

3. Real-Time Streaming:

Processing incoming orders in real time using Structured Streaming to calculate metrics such as order volume and delivery times, and to detect operational anomalies as they occur.

4. Analytics Dashboards:

Developing dashboards to visualize customer trends, restaurant performance, order patterns, and delivery efficiency.

5. Machine Learning Models:

Implementing machine learning models for demand forecasting, customer segmentation, and personalized recommendations to boost sales and improve customer experience.

6. Optimized Delivery Routing:

Using geographic data and algorithms to optimize delivery routes and reduce delivery times.

ATTRIBUTES:

DineDash will track the following key attributes across different tables:

1. CUSTOMERS

- **customer_id:** Unique identifier for each customer.
- **name:** Name of the customer.
- **email:** Email address of the customer.
- **signup_date:** Date the customer signed up.

2. RESTAURANTS

- **restaurant_id:** Unique identifier for each restaurant.
- **name:** Name of the restaurant.
- **cuisine:** Type of cuisine, such as Italian, Chinese, or Indian.
- **location_id:** Geographic location identifier, representing the latitude and longitude of the restaurant.

3. MENU ITEMS

- **item_id:** Unique identifier for each menu item.
- **restaurant_id:** Foreign key referencing the Restaurants table.

- **item_name:** Name of the menu item, such as "Margherita Pizza".
- **price:** Price of the menu item.

4. ORDERS

- **order_id:** Unique identifier for each order.
- **timestamp:** Time at which the order was placed.
- **customer_id:** Foreign key referencing the Customers table.
- **restaurant_id:** Foreign key referencing the Restaurants table.
- **agent_id:** Identifier of the delivery agent assigned to the order.
- **delivery_location_id:** Identifier representing the customer's delivery location.
- **items_ordered:** List of items included in the order, which may contain a list of menu_item_ids.
- **total_amount:** Total monetary amount of the order.
- **tip:** Tip amount provided for the order.
- **payment_method:** Payment method used, such as "Credit Card" or "Cash".
- **order_status:** Current status of the order, such as "Pending" or "Delivered".

5. DELIVERY AGENTS

- **agent_id:** Unique identifier for each delivery agent.
- **name:** Name of the delivery agent.
- **phone:** Contact phone number of the delivery agent.
- **rating:** Rating assigned to the delivery agent based on delivery performance.

Dataset Ingestion

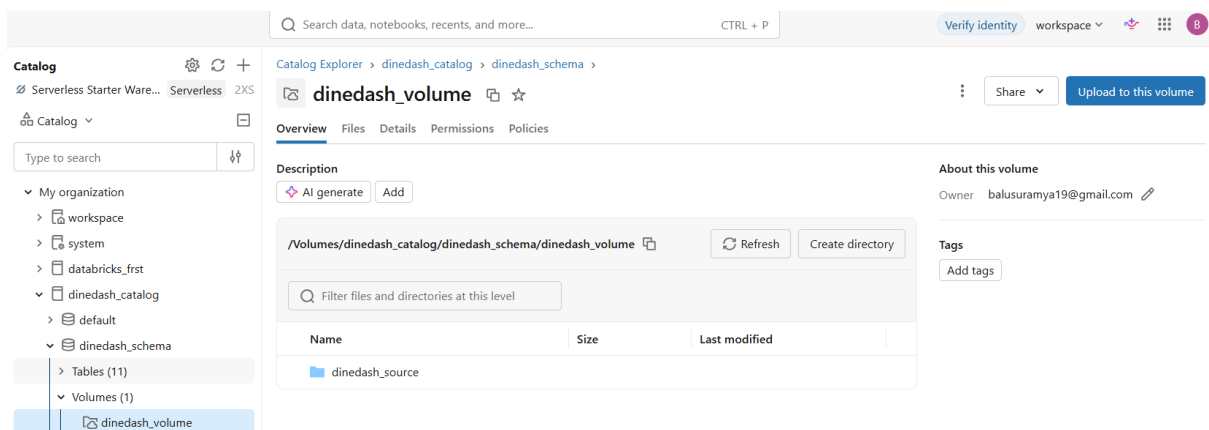
Manually Uploading datasets to DBFS In enb-accenture databricks workspace
Create folder in DBFS as/FileStore/tables//dinedosh and upload the dataset as shown below-

For orders create a folder by the name orders withinin dineodash folder and upload only Jan and Feb orders datasets to begin with

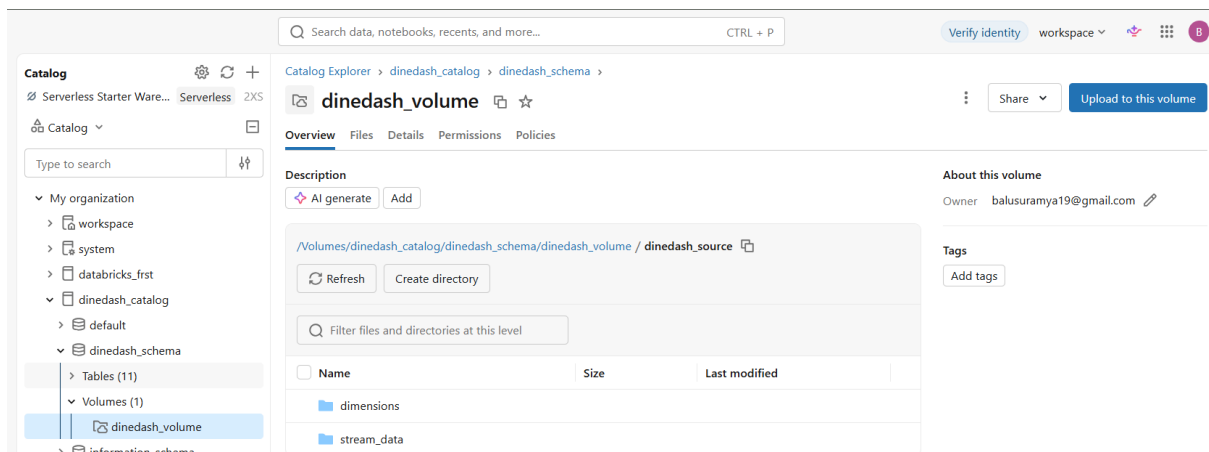
Storage Paths

The DineDash project uses a Unity Catalog Volume for centralized storage of the project data files.

dinedash_source directory created in Unity Catalog Volume (dinesdash_volume)



Two directories, dimensions and stream_data created inside the dineodash_source directory to organize the datasets.



January and February Order Datasets Ingested (Uploading Initial Datasets)

The January and February order datasets were uploaded into the `stream_data (orders)` directory.

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Catalog Explorer > dinedash_catalog > dinedash_schema >

dinedash_volume Share Upload to this volume

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume / dinedash_source / **stream_data**

Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
orders_2024_01.json	2.46 MB	4 days ago
orders_2024_02.json	2.38 MB	4 days ago

About this volume

Owner balusuramya19@gmail.com

Tags

Add tags

All dimensions datasets were uploaded into the `dimensions` directory.

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Catalog Explorer > dinedash_catalog > dinedash_schema >

dinedash_volume Share Upload to this volume

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume / dinedash_source / **dimensions**

Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
dim_customers.csv	696.52 KB	4 days ago
dim_delivery_agents.csv	17.76 KB	4 days ago
dim_locations.csv	6.18 KB	4 days ago
dim_menu_items.csv	20.08 KB	4 days ago
dim_restaurants.csv	9.53 KB	4 days ago

About this volume

Owner balusuramya19@gmail.com

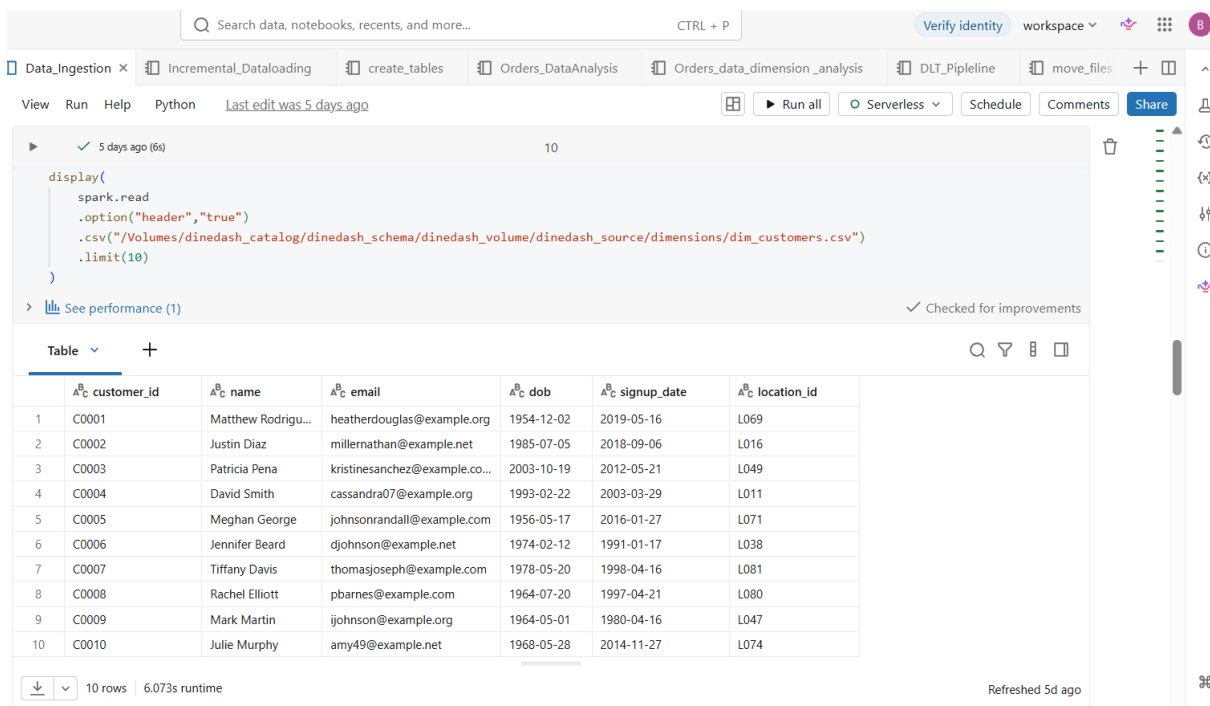
Tags

Add tags

Initial Data Exploration

Explore each of the dataset by querying against the DBFS path of each dataset files directly-Observe the sample records to understand about the dataset by displaying all the columns for 10 records

Sample records from Customer Dimension Dataset



Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Data_Ingestion x Incremental_Dataloading create_tables Orders_DataAnalysis Orders_data_dimension_analysis DLT_Pipeline move_files +

View Run Help Python Last edit was 5 days ago Run all Serverless Schedule Comments Share

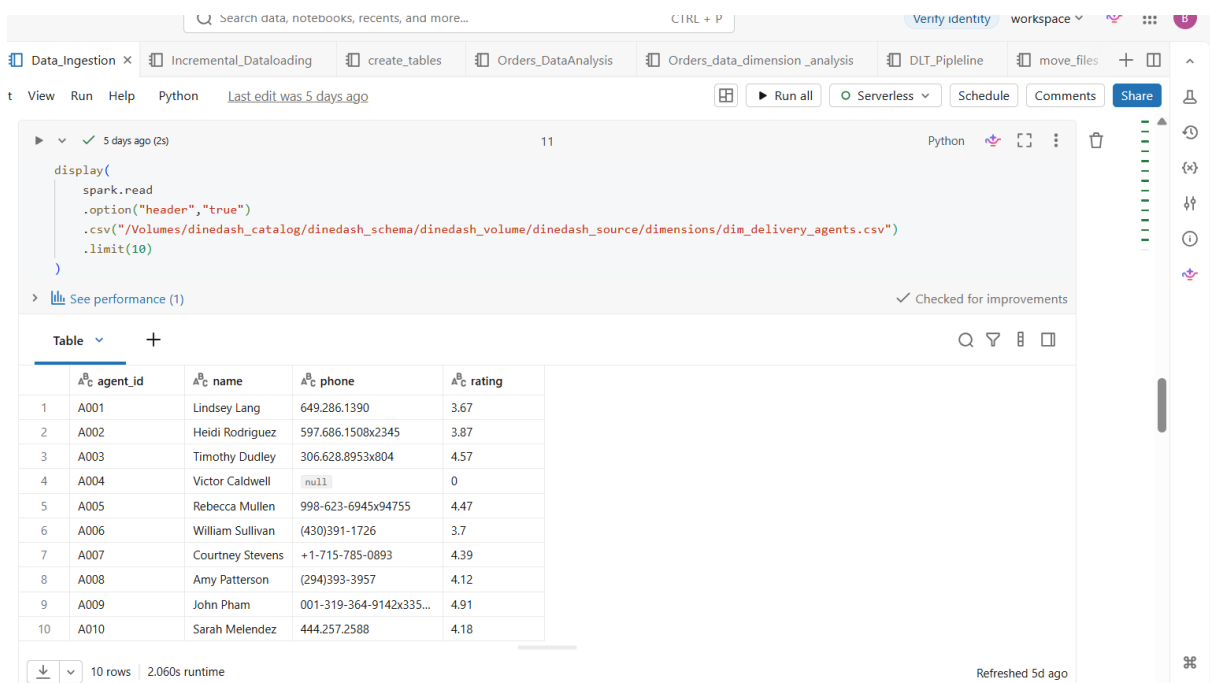
```
display(
    spark.read
      .option("header", "true")
      .csv("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_customers.csv")
      .limit(10)
)
```

See performance (1) Checked for improvements

	customer_id	name	email	dob	signup_date	location_id
1	C0001	Matthew Rodriguez	heatherdouglass@example.org	1954-12-02	2019-05-16	L069
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016
3	C0003	Patricia Pena	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071
6	C0006	Jennifer Beard	djohnson@example.net	1974-02-12	1991-01-17	L038
7	C0007	Tiffany Davis	thomasjoseph@example.com	1978-05-20	1998-04-16	L081
8	C0008	Rachel Elliott	pbarnes@example.com	1964-07-20	1997-04-21	L080
9	C0009	Mark Martin	ijohnson@example.org	1964-05-01	1980-04-16	L047
10	C0010	Julie Murphy	amy49@example.net	1968-05-28	2014-11-27	L074

10 rows 6.073s runtime Refreshed 5d ago

Sample records from Delivery Agent Dimension Dataset



Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Data_Ingestion x Incremental_Dataloading create_tables Orders_DataAnalysis Orders_data_dimension_analysis DLT_Pipeline move_files +

View Run Help Python Last edit was 5 days ago Run all Serverless Schedule Comments Share

```
display(
    spark.read
      .option("header", "true")
      .csv("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_delivery_agents.csv")
      .limit(10)
)
```

See performance (1) Checked for improvements

	agent_id	name	phone	rating
1	A001	Lindsey Lang	649.286.1390	3.67
2	A002	Heidi Rodriguez	597.686.1508x2345	3.87
3	A003	Timothy Dudley	306.628.8953x804	4.57
4	A004	Victor Caldwell	null	0
5	A005	Rebecca Mullen	998-623-6945x94755	4.47
6	A006	William Sullivan	(430)391-1726	3.7
7	A007	Courtney Stevens	+1-715-785-0893	4.39
8	A008	Amy Patterson	(294)393-3957	4.12
9	A009	John Pham	001-319-364-9142x335...	4.91
10	A010	Sarah Melendez	444.257.2588	4.18

10 rows 2.060s runtime Refreshed 5d ago

Sample records from Location Dimension Dataset

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Data_Ingestion x Incremental_Dataloading create_tables Orders_DataAnalysis Orders_data_dimension_analysis DLT_Pipeline move_files +

View Run Help Python Last edit was 5 days ago Run all Serverless Schedule Comments Share

Aug 20, 2026 (2s) 12

```
display(
    spark.read
        .option("header", "true")
        .csv("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_locations.csv")
        .limit(10)
)
```

See performance (1) Checked for improvements

	location_id	area	city	state	latitude	longitude
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126
3	L003	Daryl Extensio...	Port Justinmouth	Michigan	-19.072773	27.938702
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215
5	L005	Garcia Neck	Port Allison	Wisconsin	8.239121	26.958565
6	L006	Smith Rapids	Bobbyport	Missouri	16.931649	-49.121678
7	L007	Michael Streets	Duranport	West Virginia	-66.185224	-142.948244
8	L008	John Plains	Lake Kristinton	Oklahoma	-83.706852	28.380354
9	L009	Dennis Branch	North Brenda	Maryland	15.1168605	-67.825285
10	L010	Flores Vista	Port Joseph	Louisiana	43.557465	35.970725

10 rows 1.830s runtime Refreshed 5d ago

Sample records from Menu Items Dimension Dataset

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Data_Ingestion x Incremental_Dataloading create_tables Orders_DataAnalysis Orders_data_dimension_analysis DLT_Pipeline move_files +

View Run Help Python Last edit was 5 days ago Run all Serverless Schedule Comments Share

5 days ago (2s) 13 Python

```
display(
    spark.read
        .option("header", "true")
        .csv("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_menu_items.csv")
        .limit(10)
)
```

See performance (1) Checked for improvements

	item_id	restaurant_id	Item_name	price	category
1	I0001	R017	Radio Soda	17.97	Beverage
2	I0002	R169	Occur Pizza	27.44	Main Course
3	I0003	R178	Road Nachos	9.5	Appetizer
4	I0004	R032	Detail Juice	5.98	Beverage
5	I0005	R042	Floor Nachos	7.2	Appetizer
6	I0006	R022	Benefit Ice Cream	29.5	Dessert
7	I0007	R140	Firm Cake	9.29	Dessert
8	I0008	R077	East Pizza	6.15	Main Course
9	I0009	R137	Same Brownie	16.12	Dessert
10	I0010	R134	Wrong Juice	0	Beverage

10 rows 1.991s runtime Refreshed 5d ago

Sample records from Restaurant Dimension Dataset

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Data_Ingestion x Incremental_Dataloading create_tables Orders_DataAnalysis Orders_data_dimension_analysis DLT_Pipeline move_files +

View Run Help Python Last edit was 5 days ago Run all Serverless Schedule Comments Share

5 days ago (2s) 14 Python

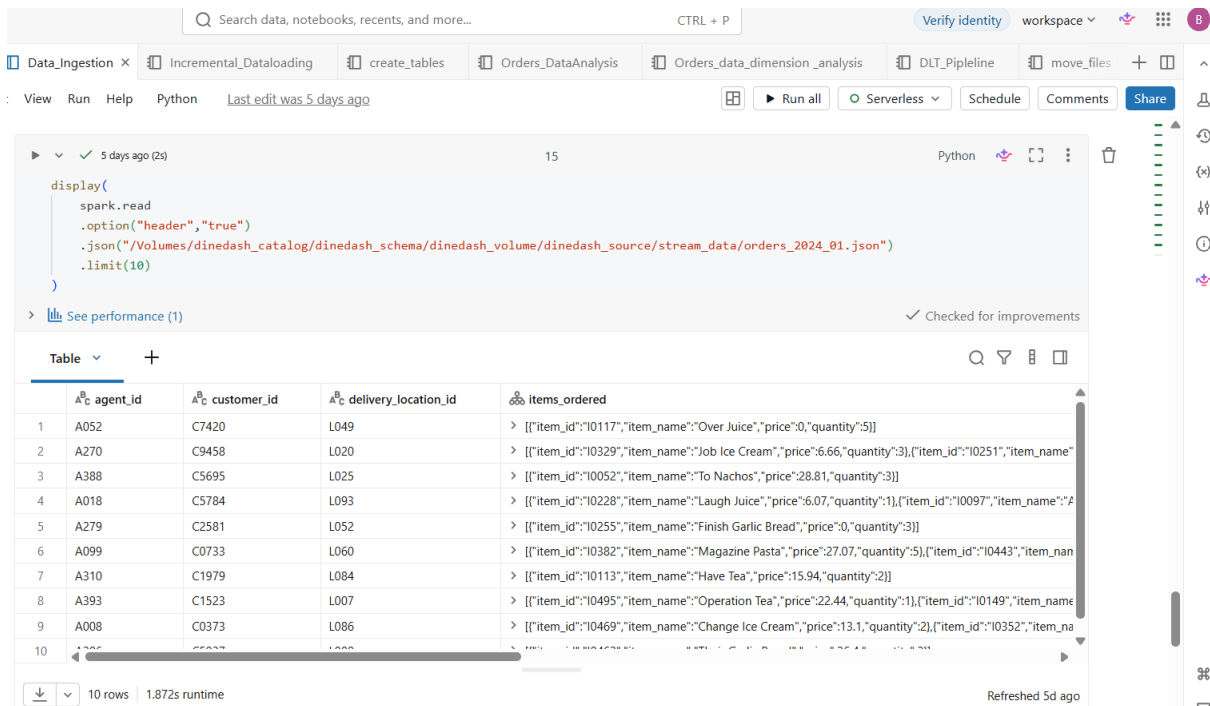
```
display(
    spark.read
        .option("header", "true")
        .csv("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_restaurants.csv")
        .limit(10)
)
```

See performance (1) Checked for improvements

	restaurant_id	name	cuisines	location_id	rating	delivery_fee
1	R001	Food Haven Grill	Indian	L069	3.3	3.9
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25
3	R003		Indian,Thai	L076	0	0
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03
5	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02
6	R006		Thai,American	L077	0	0
7	R007	Pasta Palace Corner	American,Mexican,Chinese	L014	3.3	3.33
8	R008	Curry Leaf Grill	Chinese,Mexican	L053	2.7	1.25
9	R009	Curry Leaf Corner	Mexican	L010	3.4	3.11
10	R010	Pasta Palace Hub	Mexican,Italian,American	L082	3.6	2.76

10 rows 1.552s runtime Refreshed 5d ago

Sample records from January 2024 Orders Dataset



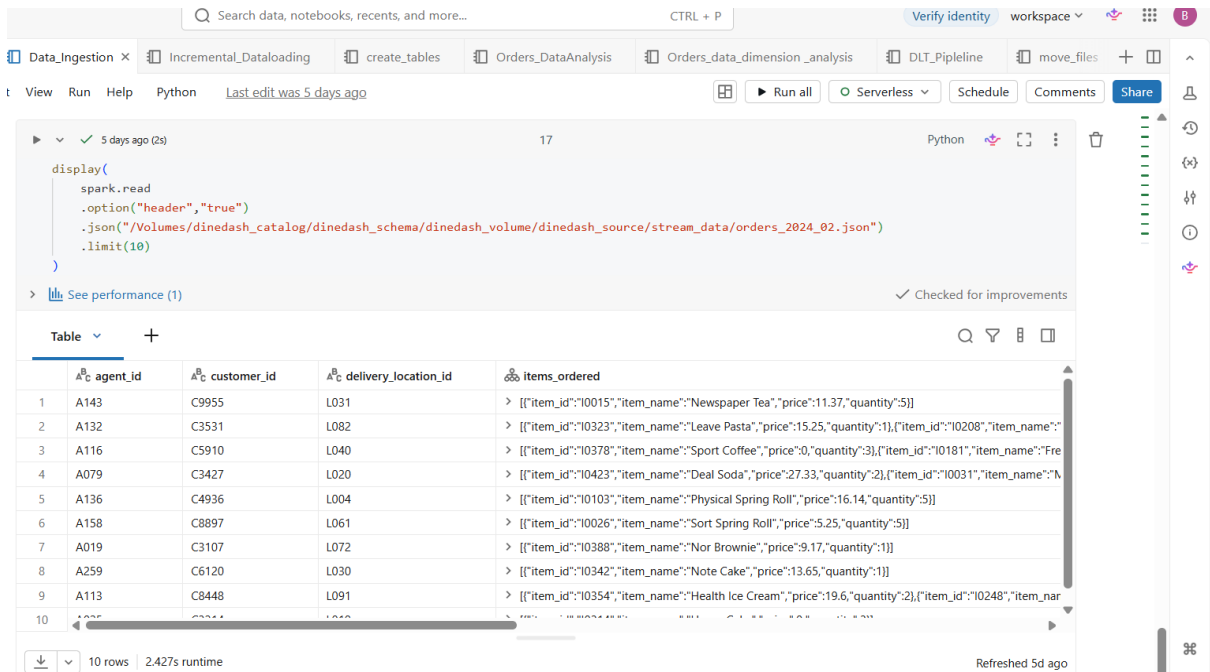
The screenshot shows a Databricks workspace interface. At the top, there's a search bar and a toolbar with buttons like 'Run all', 'Serverless', 'Schedule', 'Comments', and 'Share'. Below the toolbar, a notebook cell is visible with the following Python code:

```
display(
    spark.read
    .option("header", "true")
    .json("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/orders_2024_01.json")
    .limit(10)
)
```

The output of the cell is a table with 10 rows and 5 columns: `agent_id`, `customer_id`, `delivery_location_id`, and `items_ordered`. The `items_ordered` column contains JSON arrays of item details.

	agent_id	customer_id	delivery_location_id	items_ordered
1	A052	C7420	L049	[{"item_id": "I0117", "item_name": "Over Juice", "price": 0, "quantity": 5}]
2	A270	C9458	L020	[{"item_id": "I0329", "item_name": "Job Ice Cream", "price": 6.66, "quantity": 3}, {"item_id": "I0251", "item_name": "Job Ice Cream", "price": 6.66, "quantity": 3}]
3	A388	C5695	L025	[{"item_id": "I0052", "item_name": "To Nachos", "price": 28.81, "quantity": 3}]
4	A018	C5784	L093	[{"item_id": "I0228", "item_name": "Laugh Juice", "price": 6.07, "quantity": 1}, {"item_id": "I0097", "item_name": "Laugh Juice", "price": 6.07, "quantity": 1}]
5	A279	C2581	L052	[{"item_id": "I0255", "item_name": "Finish Garlic Bread", "price": 0, "quantity": 3}]
6	A099	C0733	L060	[{"item_id": "I0382", "item_name": "Magazine Pasta", "price": 27.07, "quantity": 5}, {"item_id": "I0443", "item_name": "Magazine Pasta", "price": 27.07, "quantity": 5}]
7	A310	C1979	L084	[{"item_id": "I0113", "item_name": "Have Tea", "price": 15.94, "quantity": 2}]
8	A393	C1523	L007	[{"item_id": "I0495", "item_name": "Operation Tea", "price": 22.44, "quantity": 1}, {"item_id": "I0149", "item_name": "Operation Tea", "price": 22.44, "quantity": 1}]
9	A008	C0373	L086	[{"item_id": "I0469", "item_name": "Change Ice Cream", "price": 13.1, "quantity": 2}, {"item_id": "I0352", "item_name": "Change Ice Cream", "price": 13.1, "quantity": 2}]
10	A008	C0373	L086	[{"item_id": "I0469", "item_name": "Change Ice Cream", "price": 13.1, "quantity": 2}, {"item_id": "I0352", "item_name": "Change Ice Cream", "price": 13.1, "quantity": 2}]

Sample records from February 2024 Orders Dataset



The screenshot shows a Databricks workspace interface. At the top, there's a search bar and a toolbar with buttons like 'Run all', 'Serverless', 'Schedule', 'Comments', and 'Share'. Below the toolbar, a notebook cell is visible with the following Python code:

```
display(
    spark.read
    .option("header", "true")
    .json("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/orders_2024_02.json")
    .limit(10)
)
```

The output of the cell is a table with 10 rows and 5 columns: `agent_id`, `customer_id`, `delivery_location_id`, and `items_ordered`. The `items_ordered` column contains JSON arrays of item details.

	agent_id	customer_id	delivery_location_id	items_ordered
1	A143	C9955	L031	[{"item_id": "I0015", "item_name": "Newspaper Tea", "price": 11.37, "quantity": 5}]
2	A132	C3531	L082	[{"item_id": "I0323", "item_name": "Leave Pasta", "price": 15.25, "quantity": 1}, {"item_id": "I0208", "item_name": "Leave Pasta", "price": 15.25, "quantity": 1}]
3	A116	C5910	L040	[{"item_id": "I0378", "item_name": "Sport Coffee", "price": 0, "quantity": 3}, {"item_id": "I0181", "item_name": "Sport Coffee", "price": 0, "quantity": 3}]
4	A079	C3427	L020	[{"item_id": "I0423", "item_name": "Deal Soda", "price": 27.33, "quantity": 2}, {"item_id": "I0031", "item_name": "Deal Soda", "price": 27.33, "quantity": 2}]
5	A136	C4936	L004	[{"item_id": "I0103", "item_name": "Physical Spring Roll", "price": 16.14, "quantity": 5}]
6	A158	C8897	L061	[{"item_id": "I0026", "item_name": "Sort Spring Roll", "price": 5.25, "quantity": 5}]
7	A019	C3107	L072	[{"item_id": "I0388", "item_name": "Nor Brownie", "price": 9.17, "quantity": 1}]
8	A259	C6120	L030	[{"item_id": "I0342", "item_name": "Note Cake", "price": 13.65, "quantity": 1}]
9	A113	C8448	L091	[{"item_id": "I0354", "item_name": "Health Ice Cream", "price": 19.6, "quantity": 2}, {"item_id": "I0248", "item_name": "Health Ice Cream", "price": 19.6, "quantity": 2}]
10	A008	C0373	L086	[{"item_id": "I0469", "item_name": "Change Ice Cream", "price": 13.1, "quantity": 2}, {"item_id": "I0352", "item_name": "Change Ice Cream", "price": 13.1, "quantity": 2}]

Creating Tables

1. Create temporary views using the `read_files()` table-valued function by inferring schema from the data for each of the datasets for customers, restaurants, locations, delivery_agents. menu_items as

customers_tmp_view

Search data, notebooks, recents, and more...

CTRL + P

Verify identityworkspace

create_tables × +

ViewRunHelpPythonLast edit was 1 minute ago

Run allServerlessScheduleCommentsShare

3 minutes ago (1s)3

%sql
CREATE OR REPLACE TEMP VIEW customers_tmp_view AS
SELECT *
from read_files(
 "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_customers.csv",
 format => "csv",
 header => "true",
 inferSchema => "true"
)
> See performance (1)

Checked for improvements

OK

1 minute ago (3s)4

SQL

%sql
select * from customers_tmp_view limit 5;
> See performance (1)

Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 5 more fields]

Table +

SearchFilterColumns

	customer_id	name	email	dob	signup_date	location_id	_rescued_data
1	C0001	Matthew Rodrigu...	heatherdougias@example.org	1954-12-02	2019-05-16	L069	null
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	null
3	C0003	Patricia Pena	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049	null
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	null
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071	null

5 rows3.356s runtime

Refreshed 1m ago

restaurants_tmp_view

Search data, notebooks, recents, and more...

CTRL + P

Verify identityworkspace

total_orders_per_restaurant × top_selling_items × daily_revenue_trends × total_revenue × DLT_Pipeline × create_tables × +

RunHelpPythonLast edit was 33 minutes ago

Run allServerlessScheduleCommentsShare

03:35 PM (1s)6

%sql
CREATE OR REPLACE TEMP VIEW restaurants_tmp_view AS
SELECT *
from read_files(
 "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_restaurants.csv",
 format => "csv",
 header => "true",
 inferSchema => "true"
)
> See performance (1)

Checked for improvements

OK

03:35 PM (5s)7

SQL

%sql
select * from restaurants_tmp_view limit 5;
> See performance (1)

Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, name: string ... 5 more fields]

Table +

SearchFilterColumns

	restaurant_id	name	cuisines	location_id	1.2 rating	1.2 delivery_fee	_rescued_data
1	R001	Food Haven G...	Indian	L069	3.3	3.9	null
2	R002	Spice Villa Hub	Thai,Italian,Chine...	L012	3.8	2.25	null
3	R003	null	Indian,Thai	L076	0	0	null
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	null
5	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02	null

5 rows4.660s runtime

Refreshed 34m ago

Locations_tmp_view

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables ×

Run Help Python Last edit was 1 minute ago

SQL

```
CREATE OR REPLACE TEMP VIEW locations_tmp_view AS
SELECT *
FROM read_files(
  "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_locations.csv",
  format => "csv",
  header => "true",
  inferSchema => "true"
)
```

See performance (1) ✓ Checked for improvements

SQL

```
select * from locations_tmp_view limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 5 more fields]

	location_id	area	city	state	latitude	longitude	_rescued_data
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	[null]
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	[null]
3	L003	Dayl Extension...	Port Judismouth	Michigan	-19.072773	27.938702	[null]
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	[null]
5	L005	Garcia Neck	Port Allison	Wisconsin	8.239121	26.958565	[null]

5 rows 10.005s runtime Refreshed Now

delivery_agents_tmp_view

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables ×

Run Help Python Last edit was 3 minutes ago

SQL

```
CREATE OR REPLACE TEMP VIEW delivery_agents_tmp_view AS
SELECT *
FROM read_files(
  "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_delivery_agents.csv",
  format => "csv",
  header => "true",
  inferSchema => "true"
)
```

See performance (1) ✓ Checked for improvements

SQL

```
select * from delivery_agents_tmp_view limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 3 more fields]

	agent_id	name	phone	rating	_rescued_data
1	A001	Lindsey Lang	649.286.1390	3.87	[null]
2	A002	Hendi Rodrigue...	597.686.1506x2345	3.87	[null]
3	A003	Timothy Dudley	306.628.8953x804	4.57	[null]
4	A004	Victor Caldwell	[null]	0	[null]
5	A005	Rebecca Mullen	998-623-6945x947...	4.47	[null]

5 rows 3.118s runtime Refreshed 3m ago

menu_items_tmp_view

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables ×

Run Help Python Last edit was now

SQL

```
CREATE OR REPLACE TEMP VIEW menu_items_tmp_view AS
SELECT *
FROM read_files(
  "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_menu_items.csv",
  format => "csv",
  header => "true",
  inferSchema => "true"
)
```

See performance (1) ✓ Checked for improvements

SQL

```
select * from menu_items_tmp_view limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 4 more fields]

	item_id	restaurant_id	item_name	price	category	_rescued_data
1	I0001	R017	Radio Soda	17.97	Beverage	[null]
2	I0002	R169	Occur Pizza	27.44	Main Course	[null]
3	I0003	R178	Road Nachos	9.5	Appetizer	[null]
4	I0004	R032	Detail Juice	5.98	Beverage	[null]
5	I0005	R042	Floor Nachos	7.2	Appetizer	[null]

5 rows 3.153s runtime Refreshed Now

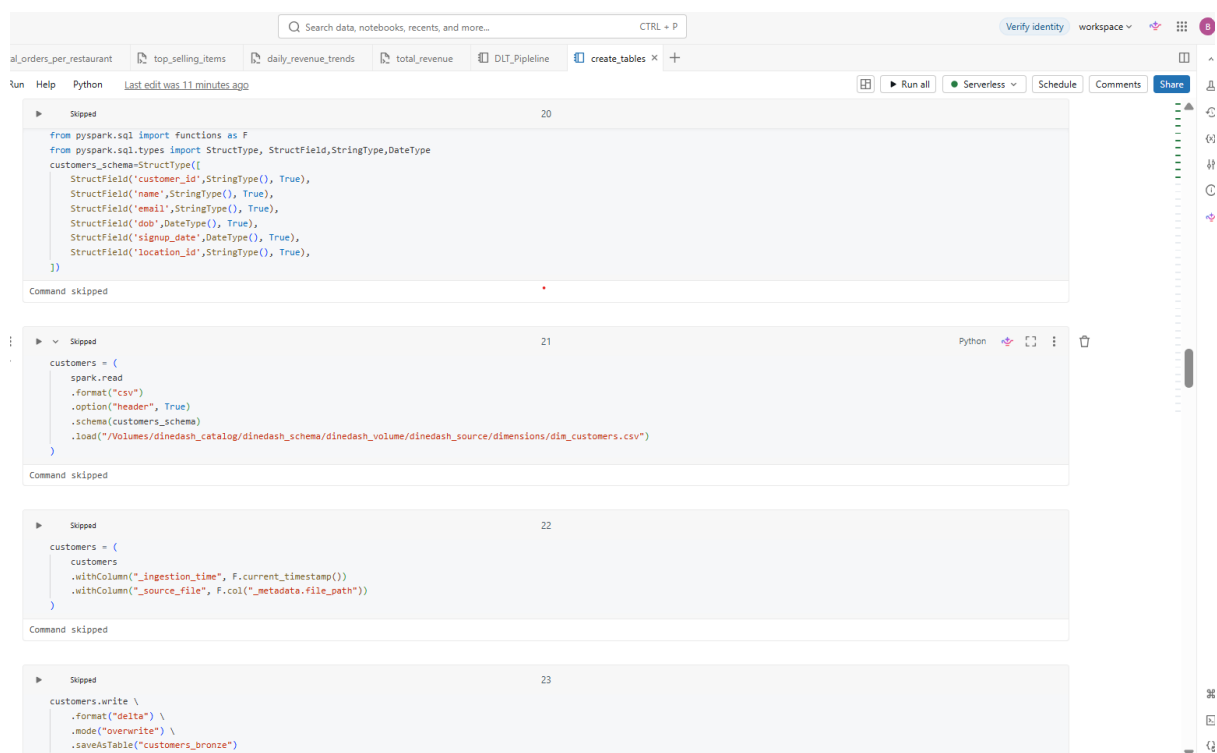
CREATING RAW DELTA TABLES

2. Create raw delta tables providing the manual schema and capture the record ingestion time and source file name for each record. Create tables for each dataset.

Ensure that all tables have appropriate column names and data types

Creation of Bronze Delta tables using manually defined schemas with ingestion timestamp and source file metadata.

customers_bronze

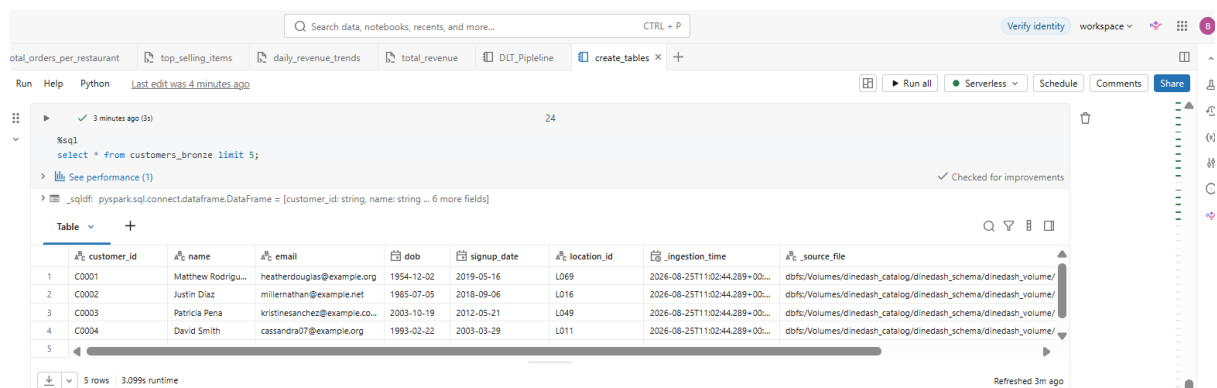


```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DateType

customers_schema=StructType([
    StructField('customer_id',StringType(), True),
    StructField('name',StringType(), True),
    StructField('email',StringType(), True),
    StructField('dob',DateType(), True),
    StructField('signup_date',DateType(), True),
    StructField('location_id',StringType(), True),
])

customers = (
    spark.read
    .format("csv")
    .option("header", True)
    .schema(customers_schema)
    .load("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_customers.csv")
)

customers \
    .withColumn("ingestion_time", F.current_timestamp()) \
    .withColumn("source_file", F.col("_metadata.file_path")) \
    .write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("customers_bronze")
```



```
%sql
select * from customers_bronze limit 5;
```

customer_id	name	email	dob	signup_date	location_id	ingestion_time	source_file
C0001	Matthew Rodriguez	heatherdougias@example.org	1954-12-02	2019-05-16	L069	2026-08-25T11:02:44.289+00...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/
C0002	Justin Diaz	milliemathan@example.net	1985-07-05	2018-09-06	L016	2026-08-25T11:02:44.289+00...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/
C0003	Patricia Pena	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049	2026-08-25T11:02:44.289+00...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/
C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-25T11:02:44.289+00...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/

restaurants_bronze

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables x +

View Run Help Python Last edit was 6 minutes ago Run all Serverless Schedule Comments Share

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
restaurants_schema=StructType([
    StructField("restaurant_id", StringType(), True),
    StructField("name", StringType(), True),
    StructField("cuisines", StringType(), True),
    StructField("location_id", StringType(), True),
    StructField("rating", DoubleType(), True),
    StructField("delivery_fee", DoubleType(), True),
])

restaurants=(
    spark.read
    .format("csv")
    .option("header", True)
    .schema(restaurants_schema)
    .load("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_restaurants.csv")
    .withColumn("_ingestion_time", F.current_timestamp())
    .withColumn("_source_file", F.col("_metadata.file_path"))
)

restaurants.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("restaurants_bronze")
```

See performance (1) ✓ Checked for improvements

restaurants: pyspark.sql.connect.dataframe.DataFrame = (restaurant_id: string, name: string ... 6 more fields)

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables x +

View Run Help Python Last edit was now Run all Serverless Schedule Comments Share

```
%sql
select * from restaurants_bronze limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlid: pyspark.sql.connect.dataframe.DataFrame = (restaurant_id: string, name: string ... 6 more fields)

	restaurant_id	name	cuisines	location_id	rating	delivery_fee	_ingestion_time	_source_file
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	2026-08-25T11:08:32.363+00...	dbfs:/Volumes/dinedash_catalog/din...
2	R002	Spice Villa Hub	Thai,Italian,Chine...	L012	3.8	2.25	2026-08-25T11:08:32.363+00...	dbfs:/Volumes/dinedash_catalog/din...
3	R003	null	Indian,Thai	L076	0	0	2026-08-25T11:08:32.363+00...	dbfs:/Volumes/dinedash_catalog/din...
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	2026-08-25T11:08:32.363+00...	dbfs:/Volumes/dinedash_catalog/din...
5								

5 rows 1.790s runtime Refreshed Now

locations_bronze

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables x +

View Run Help Python Last edit was 2 minutes ago Run all Serverless Schedule Comments Share

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
location_schema=StructType([
    StructField("location_id", StringType(), True),
    StructField("area", StringType(), True),
    StructField("city", StringType(), True),
    StructField("state", StringType(), True),
    StructField("latitude", DoubleType(), True),
    StructField("longitude", DoubleType(), True),
])

locations=(spark.read
    .format("csv")
    .option("header", True)
    .schema(location_schema)
    .load("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_locations.csv")
    .withColumn("_ingestion_time", F.current_timestamp())
    .withColumn("_source_file", F.col("_metadata.file_path"))
)

locations.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("locations_bronze")
```

See performance (1)

locations: pyspark.sql.connect.dataframe.DataFrame = (location_id: string, area: string ... 6 more fields)

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables x +

ew Run Help Python Last edit was now Run all Serverless Schedule Comments Share

Just now (2s) 34 SQL

```
%sql
select * from locations_bronze limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 6 more fields]

	location_id	area	city	state	latitude	longitude	ingestion_time	source_file
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
3	L003	Daryl Extensio...	Port Justinmouth	Michigan	-19.072773	27.938702	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
5								

5 rows 2.415s runtime Refreshed Now

delivery_agents_bronze

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables x +

View Run Help Python Last edit was 2 minutes ago Run all Serverless Schedule Comments Share

Just now (<1s) 37

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
delivery_schema=StructType([
    StructField('agent_id', StringType(), True),
    StructField('name', StringType(), True),
    StructField('phone', StringType(), True),
    StructField('rating', DoubleType(), True),
])
```

See performance (1) ✓ Checked for improvements

_delivery_agents: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 4 more fields]

Just now (5s) 38 Python

```
delivery_agents=(spark.read
    .format("csv")
    .option("header",True)
    .schema(delivery_schema)
    .load("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_delivery_agents.csv")
    .withColumn("_ingestion_time", F.current_timestamp())
    .withColumn("_source_file", F.col("_metadata.file_path"))
)
delivery_agents.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("delivery_agents_bronze")
```

See performance (1) ✓ Checked for improvements

_delivery_agents: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 4 more fields]

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables x +

ew Run Help Python Last edit was now Run all Serverless Schedule Comments Share

Just now (2s) 39 SQL

```
%sql
select * from delivery_agents_bronze limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 4 more fields]

	agent_id	name	phone	rating	ingestion_time	source_file
1	A001	Lindsey Lang	649.286.1390	3.67	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
2	A002	Heidi Rodrigu...	597.686.1508x2345	3.87	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
3	A003	Timothy Dudley	306.628.8953x804	4.57	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
4	A004	Victor Caldwell	null	0	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
5						

5 rows 2.008s runtime Refreshed Now

menu items bronze

Search data, notebooks, recents, and more... CTRL + P

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables × +

view Run Help Python Last edit was 1 minute ago

Run all Serverless Schedule Comments Share

```
42
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
menuitems_schema = StructType([
    StructField('item_id', StringType(), True),
    StructField('restaurant_id', StringType(), True),
    StructField('item_name', StringType(), True),
    StructField('price', DoubleType(), True),
    StructField('category', StringType(), True),
])

43
menuitems = (spark.read
    .format("csv")
    .option("header", True)
    .schema(menuitems_schema)
    .load("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_menu_items.csv")
    .withColumn("_ingestion_time", F.current_timestamp())
    .withColumn("_source_file", F.col("_metadata.file_path")))
menuitems.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("Menu_items_bronze")
```

See performance (1) ✓ Checked for improvements

menuitems: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 5 more fields]

Search data, notebooks, recents, and more... CTRL + P

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables × +

View Run Help Python Last edit was 3 minutes ago

Run all Serverless Schedule Comments Share

```
16
%sql
select * from menu_items_tmp_view limit 5;
```

See performance (1) ✓ Checked for improvements

Table	Item_id	restaurant_id	Item_name	price	category	rescued_data
1	I0001	R017	Radio Soda	17.97	Beverage	null
2	I0002	R169	Occur Pizza	27.44	Main Course	null
3	I0003	R178	Road Nachos	9.5	Appetizer	null
4	I0004	R032	Detail Juice	5.98	Beverage	null
5	I0005	R042	Floor Nachos	7.2	Appetizer	null

5 rows 3.153s runtime Refreshed 23m ago

CREATE CLEANED DATA DELTA TABLES (FILTERED)

Customers_filtered

Search data, notebooks, recents, and more... CTRL + P

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables × +

view Run Help Python Last edit was 3 minutes ago

Run all Serverless Schedule Comments Share

```
46
customers_filtered = (spark.table("customers_bronze")
    .filter(
        F.col("customer_id").isNotNull() &
        F.col("name").isNotNull() &
        F.col("location_id").isNotNull() &
        F.col("signup_date").isNotNull() &
        F.col("dob").isNotNull() &
        F.col("email").isNotNull()
    )
)
customers_filtered.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("customers_filtered")
```

See performance (1) ✓ Checked for improvements

customers_filtered: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 6 more fields]

Which contains the records where customer_id, signup_date, dob, email, name are not missing

Just now (2s) 47

```
%sql
select * from customers_filtered limit 5;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 6 more fields]

	customer_id	name	email	dob	signup_date	location_id	ingestion_time	source_file
1	C0001	Matthew Rodrigu...	heatherdouglas@example.org	1954-12-02	2019-05-16	L069	2026-08-25T11:02:44.289+00:...	dbfs:/Volumes/dinedash_c
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	2026-08-25T11:02:44.289+00:...	dbfs:/Volumes/dinedash_c
3	C0003	Patricia Pena	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049	2026-08-25T11:02:44.289+00:...	dbfs:/Volumes/dinedash_c
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-25T11:02:44.289+00:...	dbfs:/Volumes/dinedash_c
5								

5 rows | 2.182s runtime Refreshed Now

restaurants filtered

Search data, notebooks, reents, and more... CTRL + P

Verify identity workspace

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables × +

View Run Help Python Last edit was now

Run all Serverless Schedule Comments Share

Just now (4s) 49

```
restaurants_filtered = (spark.table("restaurants_bronze")
    .filter(
        F.col("restaurant_id").isNotNull() &
        F.col("name").isNotNull() &
        F.col("cuisines").isNotNull() &
        F.col("location_id").isNotNull() &
        F.col("rating").isNotNull() &
        F.col("delivery_fee").isNotNull()
    )
)
restaurants_filtered.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("restaurants_filtered")
```

See performance (1) ✓ Checked for improvements

restaurants_filtered: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, name: string ... 6 more fields]

Which contains the records where restaurant_id, name, cuisines, location_id, rating, delivery_fee are not missing.

Just now (2s) 50

```
%sql
select * from restaurants_filtered limit 5;
```

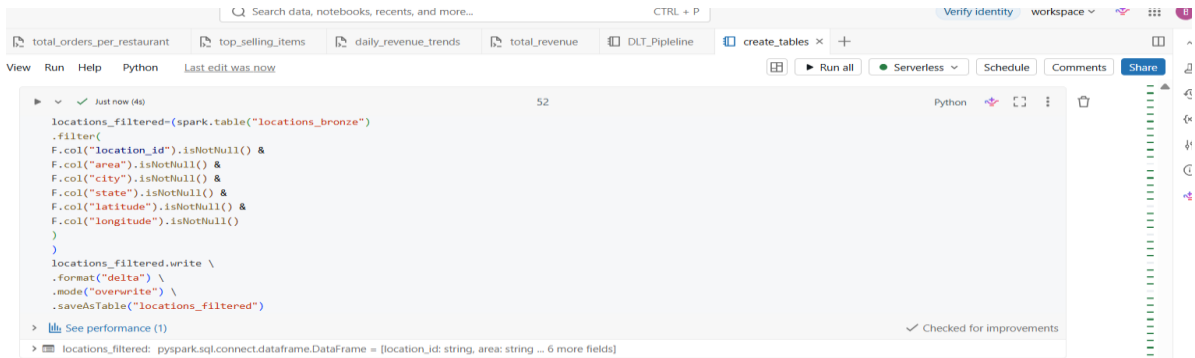
See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, name: string ... 6 more fields]

	restaurant_id	name	cuisines	location_id	rating	delivery_fee	ingestion_time	source_file
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	2026-08-25T11:08:32.363+00:...	dbfs:/Volumes/dinedash_c
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25	2026-08-25T11:08:32.363+00:...	dbfs:/Volumes/dinedash_c
3	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	2026-08-25T11:08:32.363+00:...	dbfs:/Volumes/dinedash_c
4	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02	2026-08-25T11:08:32.363+00:...	dbfs:/Volumes/dinedash_c
5								

5 rows | 2.205s runtime Refreshed Now

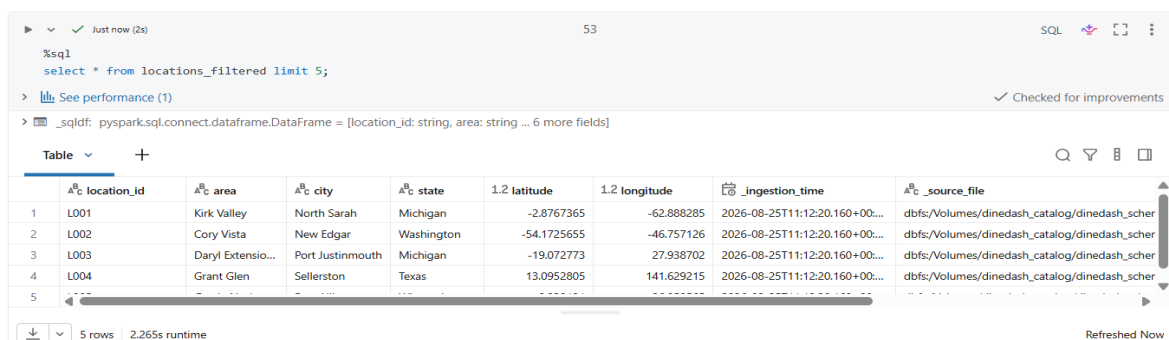
locations filtered



```
locations_filtered=(spark.table("locations_bronze")
.filter(
  F.col("location_id").isNotNull() &
  F.col("area").isNotNull() &
  F.col("city").isNotNull() &
  F.col("state").isNotNull() &
  F.col("latitude").isNotNull() &
  F.col("longitude").isNotNull()
)
)
locations_filtered.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("locations_filtered")
```

locations_filtered: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 6 more fields]

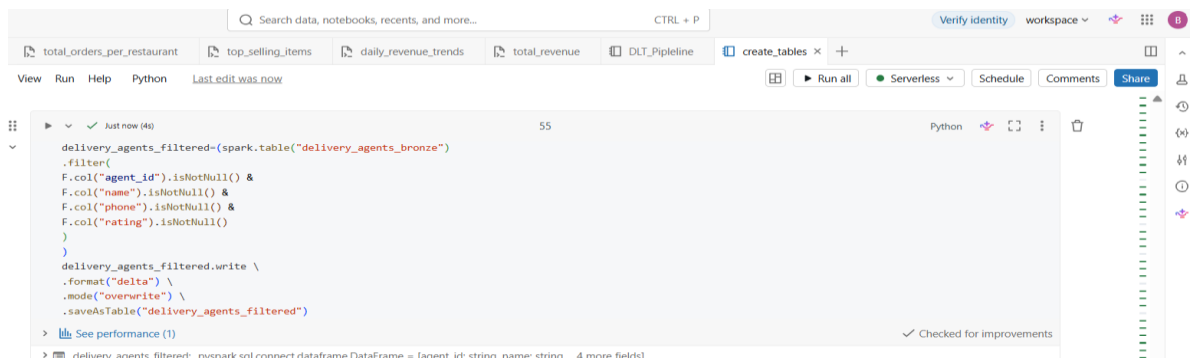
Which contains the records where location_id, city, state, latitude, longitude are not missing



```
%sql
select * from locations_filtered limit 5;
```

	location_id	area	city	state	latitude	longitude	ingestion_time	source_file
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
3	L003	Daryl Extensio...	Port Justinmouth	Michigan	-19.072773	27.938702	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	2026-08-25T11:12:20.160+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_scher

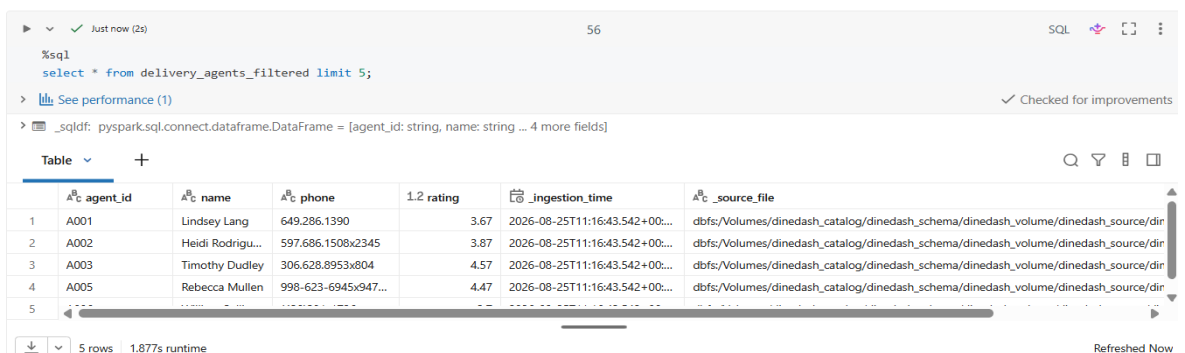
Delivery_agents_filtered



```
delivery_agents_filtered=(spark.table("delivery_agents_bronze")
.filter(
  F.col("agent_id").isNotNull() &
  F.col("name").isNotNull() &
  F.col("phone").isNotNull() &
  F.col("rating").isNotNull()
)
)
delivery_agents_filtered.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("delivery_agents_filtered")
```

delivery_agents_filtered: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 4 more fields]

Which contains the records where agent_id, name, rating are not missing



```
%sql
select * from delivery_agents_filtered limit 5;
```

	agent_id	name	phone	rating	ingestion_time	source_file
1	A001	Lindsey Lang	649.286.1390	3.67	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
2	A002	Heidi Rodrigu...	597.686.1508x2345	3.87	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
3	A003	Timothy Dudley	306.628.8953x804	4.57	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir
4	A005	Rebecca Mullen	998-623-6945x947...	4.47	2026-08-25T11:16:43.542+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dir

menu items filtered

Search data, notebooks, recents, and more... CTRL + P

total_orders_per_restaurant top_selling_items daily_revenue_trends total_revenue DLT_Pipeline create_tables × +

View Run Help Python Last edit was now Interrupt Serverless Schedule Comments Share

```
Just now (4s) 58 Python
menu_items_filtered=(spark.table("menu_items_bronze")
    .filter(
        F.col("item_id").isNull() &
        F.col("restaurant_id").isNull() &
        F.col("item_name").isNull() &
        F.col("price").isNull() &
        F.col("category").isNull()
    ))
menu_items_filtered.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("menu_items_filtered")
> See performance (1)
menu_items_filtered: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 5 more fields]
```

which contains the records where Item_Id, name, price, category are not missing

Just now (2s) 59 SQL

```
%sql
select * from menu_items_filtered limit 5;
> See performance (1)
_sqlcdf: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 5 more fields]
```

	item_id	restaurant_id	item_name	price	category	ingestion_time	source_file
1	I0001	R017	Radio Soda	17.97	Beverage	2026-08-25T11:18:26.053+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_v
2	I0002	R169	Occur Pizza	27.44	Main Course	2026-08-25T11:18:26.053+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_v
3	I0003	R178	Road Nachos	9.5	Appetizer	2026-08-25T11:18:26.053+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_v
4	I0004	R032	Detail Juice	5.98	Beverage	2026-08-25T11:18:26.053+00:...	dbfs:/Volumes/dinedash_catalog/dinedash_schema/dinedash_v
5							

5 rows | 1.773s runtime Refreshed Now

Record count comparison between Bronze and Filtered tables

Just now (4s) 60 Python

```
display(spark.sql("""
SELECT 'customers' AS table, COUNT(*) AS bronze, (SELECT COUNT(*) FROM customers_filtered) AS filtered FROM customers_bronze
UNION ALL SELECT 'restaurants', COUNT(*), (SELECT COUNT(*) FROM restaurants_filtered) FROM restaurants_bronze
UNION ALL SELECT 'locations', COUNT(*), (SELECT COUNT(*) FROM locations_filtered) FROM locations_bronze
UNION ALL SELECT 'delivery_agents', COUNT(*), (SELECT COUNT(*) FROM delivery_agents_filtered) FROM delivery_agents_bronze
UNION ALL SELECT 'menu_items', COUNT(*), (SELECT COUNT(*) FROM menu_items_filtered) FROM menu_items_bronze
"""))
> See performance (1)
```

	table	bronze	filtered
1	customers	10050	9999
2	restaurants	200	150
3	locations	100	100
4	delivery_age...	450	395
5	menu_items	500	500

5 rows | 4.411s runtime Refreshed Now

Incremental Data Loading

Create a table named `orders_bronze` without schema and incrementally load order details for each month of the year 2024, Ingesting one month's data at a time using `COPY INTO`

Ingest January, February month order details, verify the records count and then Ingest March month order details, verify the records count

Create `orders_bronze` table

The screenshot shows a Databricks notebook interface. The top bar includes a search bar, 'Verify identity', 'workspace', and a user profile icon. The notebook title is 'Incremental_DataLoading'. Below the title bar, there are tabs for 'View', 'Run', 'Help', 'Python', and a link to 'Last edit was yesterday'. The main code cell contains a SQL command to create a table named `orders_bronze` using the `delta` storage format. The command is: `%sql create table orders_bronze using delta;`. The output shows a successful execution with a message 'Checked for improvements' and a 'See performance (1)' link. The status bar at the bottom indicates 'OK'.

```
%sql
create table orders_bronze
using delta;
```

Incremental ingestion of January and February 2024 order data using `COPY INTO`

The screenshot shows a Databricks notebook interface. The top bar includes a search bar, 'Verify identity', 'workspace', and a user profile icon. The notebook title is 'Incremental_DataLoading'. Below the title bar, there are tabs for 'Data_Ingestion', 'create_tables', and 'Incremental_DataLoading'. The main code cell contains a SQL command to ingest data into the `orders_bronze` table using the `COPY INTO` command. The command is: `%sql COPY INTO orders_bronze FROM "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/" FILEFORMAT = JSON COPY_OPTIONS ('mergeSchema' = 'true');`. The output shows a successful execution with a message 'Checked for improvements' and a 'See performance (1)' link. Below the code cell, there is a table showing the results of the ingestion. The table has four columns: `num_affected_rows`, `num_inserted_rows`, `num_skipped_corrupt_files`, and an unnamed column. The data row shows 1 row affected, 11632 rows inserted, 0 corrupt files skipped, and 0 in the unnamed column.

```
%sql
COPY INTO orders_bronze
FROM "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/"
FILEFORMAT = JSON
COPY_OPTIONS ('mergeSchema' = 'true');
```

	<code>num_affected_rows</code>	<code>num_inserted_rows</code>	<code>num_skipped_corrupt_files</code>	
1	11632	11632	0	

January, February record count

The screenshot shows a Databricks notebook interface. The top bar includes a search bar, 'Verify identity', 'workspace', and a user profile icon. The notebook title is 'Incremental_DataLoading'. Below the title bar, there are tabs for 'View', 'Run', 'Help', 'Python', and a link to 'Last edit was yesterday'. The main code cell contains a SQL command to select the count of records from the `orders_bronze` table. The command is: `%sql SELECT COUNT(*) AS total_records FROM orders_bronze;`. The output shows a successful execution with a message 'Checked for improvements' and a 'See performance (1)' link. Below the code cell, there is a table showing the results of the query. The table has two columns: `total_records` and an unnamed column. The data row shows 1 row with a total of 11632 records.

```
%sql
SELECT COUNT(*) AS total_records
FROM orders_bronze;
```

	<code>total_records</code>
1	11632

Ingestion of March 2024 order data using COPY INTO

The screenshot shows a Databricks notebook with a SQL cell that has been executed. The SQL code is as follows:

```
%sql
COPY INTO orders_bronze
FROM "/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/"
FILEFORMAT = JSON
COPY_OPTIONS ('mergeSchema' = 'true');
```

Below the code, the execution results are displayed in a table format:

	num_affected_rows	num_inserted_rows	num_skipped_corrupt_files
1	5019	5019	0

January, February and March record count

The screenshot shows a Databricks notebook with a SQL cell that has been executed. The SQL code is as follows:

```
%sql
SELECT COUNT(*) AS total_records
FROM orders_bronze;
```

Below the code, the execution results are displayed in a table format:

	total_records
1	16651

ORDER DATA ANALYSIS

1. Total number of orders placed.

The screenshot shows a Databricks notebook with a SQL cell that has been executed. The SQL code is as follows:

```
%sql
SELECT COUNT(*) AS total_orders
FROM orders_bronze;
```

Below the code, the execution results are displayed in a table format:

	total_orders
1	16651

2. Distribution of orders by payment method

The screenshot shows a Databricks notebook with a SQL cell that has been executed. The SQL code is as follows:

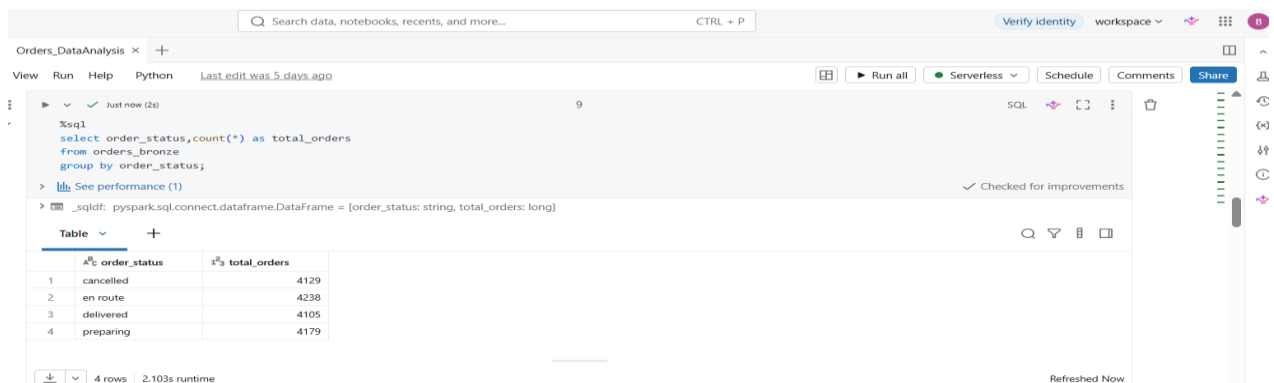
```
%sql
select payment_method, count(*) as total_orders
from orders_bronze
group by payment_method;
```

Below the code, the execution results are displayed in a table format:

	payment_method	total_orders
1	card	4186
2	UPI	4124
3	wallet	4140
4	cash	4201

At the bottom of the screenshot, it indicates "4 rows" and "8.183s runtime".

3. Distribution of order status (e.g., delivered, preparing, cancelled)



Orders_DataAnalysis × +

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

View Run Help Python Last edit was 5 days ago

Run all Serverless Schedule Comments Share

Just now (2s) 9

SQL

```
%sql
select order_status,count(*) as total_orders
from orders_bronze
group by order_status;
```

See performance (1)

Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [order_status: string, total_orders: long]

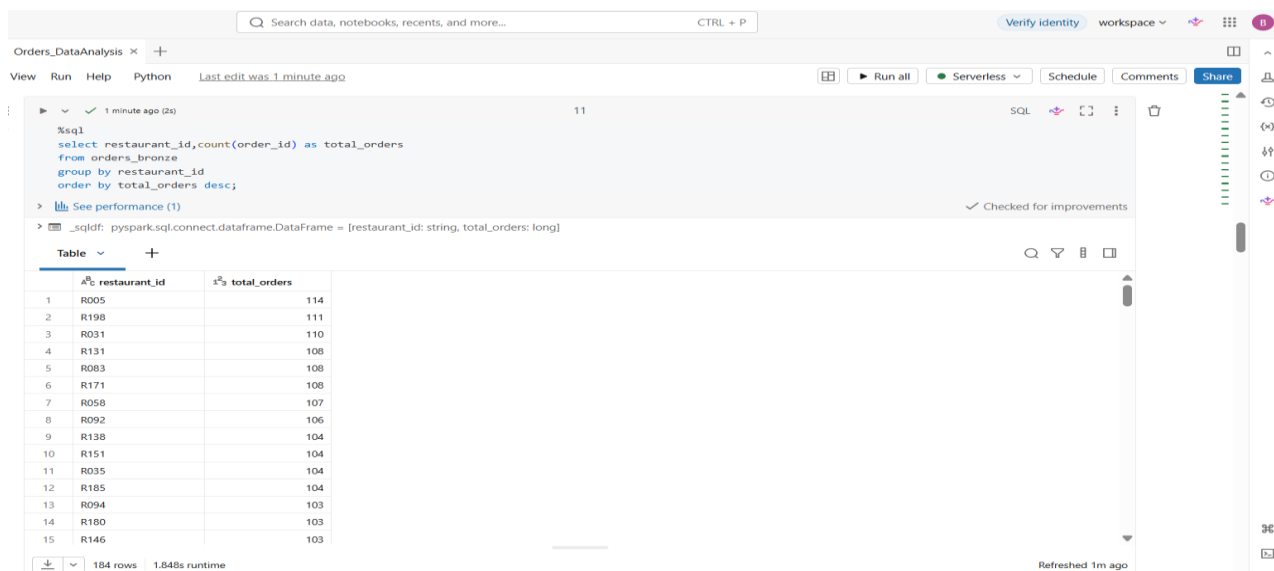
Table +

	order_status	total_orders
1	cancelled	4129
2	en route	4238
3	delivered	4105
4	preparing	4179

4 rows 2.103s runtime

Refreshed Now

4. Orders per restaurant (volume of business)



Orders_DataAnalysis × +

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

View Run Help Python Last edit was 1 minute ago

Run all Serverless Schedule Comments Share

1 minute ago (2s) 11

SQL

```
%sql
select restaurant_id,count(order_id) as total_orders
from orders_bronze
group by restaurant_id
order by total_orders desc;
```

See performance (1)

Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, total_orders: long]

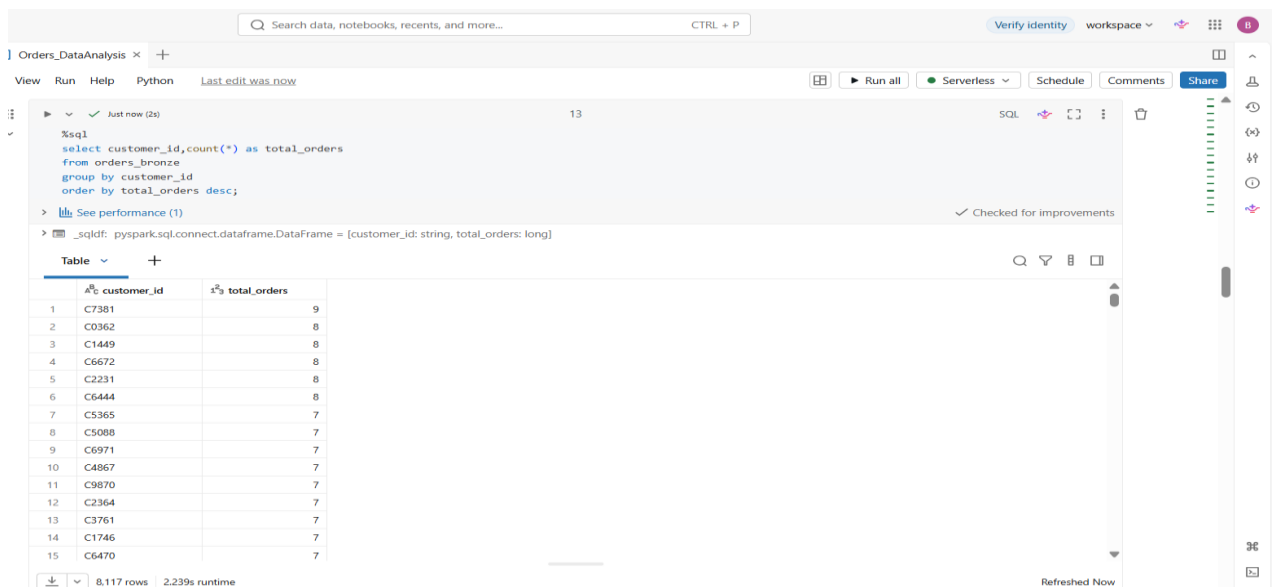
Table +

	restaurant_id	total_orders
1	R005	114
2	R198	111
3	R031	110
4	R131	108
5	R083	108
6	R171	108
7	R058	107
8	R092	106
9	R138	104
10	R151	104
11	R035	104
12	R185	104
13	R094	103
14	R180	103
15	R146	103

184 rows 1.848s runtime

Refreshed 1m ago

5. Orders per customer (loyalty insight)



Orders_DataAnalysis × +

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

View Run Help Python Last edit was now

Run all Serverless Schedule Comments Share

Just now (2s) 13

SQL

```
%sql
select customer_id,count(*) as total_orders
from orders_bronze
group by customer_id
order by total_orders desc;
```

See performance (1)

Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, total_orders: long]

Table +

	customer_id	total_orders
1	C7381	9
2	C0362	8
3	C1449	8
4	C6672	8
5	C2231	8
6	C6444	8
7	C5365	7
8	C5088	7
9	C6971	7
10	C4867	7
11	C9870	7
12	C2364	7
13	C3761	7
14	C1746	7
15	C6470	7

8,117 rows 2.239s runtime

Refreshed Now

6. Orders handled per delivery agent

Orders_DataAnalysis x +

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

View Run Help Python Last edit was now

Run all Serverless Schedule Comments Share

Just now (2s) 15

```
%sql
select agent_id, count(order_id) as total_orders
from orders_bronze
group by agent_id
order by total_orders desc;
```

See performance (1) Checked for improvements

_sqlid: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, total_orders: long]

	1 agent_id	2 total_orders
1	A309	117
2	A207	114
3	A393	99
4	A350	91
5	A138	88
6	A245	86
7	A168	86
8	A315	84
9	A370	83
10	A056	81
11	A230	81
12	A360	79
13	A178	79
14	A061	79
15	A288	78

400 rows 1.775s runtime Refreshed Now

7. Top 10 orders by total amount

Orders_DataAnalysis x +

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

View Run Help Python Last edit was 5 minutes ago

Run all Serverless Schedule Comments Share

4 minutes ago (2s) 17

```
%sql
select order_id, total_amount
from orders_bronze
order by total_amount desc
limit 10;
```

See performance (1) Checked for improvements

_sqlid: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, total_amount: double]

	1 order_id	2 total_amount
1	c12860c3-683a-47f7-b790-7cbbbcf0fec	361.46
2	05e56a7c-190c-4bde-ac13-e34b38fe155a	355.9
3	341cb220-46e4-4f0d-b2f2-2e494b5729dd	352.45
4	3b6f5bd2-08e4-412d-be9e-7701463015...	348.95
5	f43228bd-94e5-4ef6-914a-c7ebdceaa572	342.91
6	621d746e-bd7d-4e27-b06e-a098a9bffd7	340.65
7	75dfeba8-60e0-42c2-9149-315f030e8d9d	333.85
8	72559328-7d0d-4177-8c44-4ebc43dca731	332.98
9	b9dc92fc-1922-4274-a42c-85b83e46bea2	327.8
10	7d9b6852-ffb2-4bac-84e4-cc7960284e30	327.44

10 rows 1.877s runtime Refreshed 4m ago

8. Average tip amount per payment method

Orders_DataAnalysis x +

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

View Run Help Python Last edit was 6 minutes ago

Run all Serverless Schedule Comments Share

Just now (2s) 19

```
%sql
select payment_method, round(avg(tip), 2) as average_tip_amount
from orders_bronze
group by payment_method
```

See performance (1) Checked for improvements

_sqlid: pyspark.sql.connect.dataframe.DataFrame = [payment_method: string, average_tip_amount: double]

	1 payment_method	2 average_tip_amount
1	card	2.51
2	UPI	2.53
3	wallet	2.5
4	cash	2.47

4 rows 1.756s runtime Refreshed Now

9. Percentage of orders with tip > 10% of total_amount

The screenshot shows a Databricks notebook interface. The top bar includes a search bar, a workspace dropdown, and a user profile icon. The notebook title is "Orders_DataAnalysis". The code cell contains a SQL query that calculates the percentage of orders where the tip is greater than 10% of the total amount. The query is as follows:

```
%sql
select round(sum(case when tip > total_amount*0.10 then 1 else 0 end)*100.0/count(*),2)as percentage_orders
from orders_bronze;
```

The output of the query is a single row with the value 21.49.

.00 percentage_orders
21.49

10. Percentage of successful vs cancelled vs failed orders

The screenshot shows a Databricks notebook interface. The top bar includes a search bar, a workspace dropdown, and a user profile icon. The notebook title is "Orders_DataAnalysis". The code cell contains a SQL query that calculates the percentage of orders for each status (en route, preparing, cancelled, delivered). The query is as follows:

```
%sql
select order_status,round(count(*)*100.0/sum(count(*) OVER(),2) as percentage_of_orders
from orders_bronze
group by order_status
order by percentage_of_orders desc;
```

The output of the query is a table with 4 rows and 2 columns: order_status and percentage_of_orders.

order_status	.00 percentage_of_orders
en route	25.45
preparing	25.10
cancelled	24.80
delivered	24.65

11. Top customers (by number of orders) and top restaurants (by number of orders).

The screenshot shows a Databricks notebook interface. The top bar includes a search bar, a workspace dropdown, and a user profile icon. The notebook title is "Orders_DataAnalysis". The code cell contains a SQL query that finds the top 10 customers and top 10 restaurants by number of orders. The query is as follows:

```
%sql
SELECT customer_id,count(*) AS total_orders
FROM orders_bronze
GROUP BY customer_id
ORDER BY total_orders DESC
LIMIT 10;
```

The output of the query is a table with 10 rows and 2 columns: customer_id and total_orders.

customer_id	total_orders
C7381	9
C2231	8
C0362	8
C6672	8
C6444	8
C1449	8
C7067	7
C1423	7
C9870	7
C0313	7

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

Orders_DataAnalysis

View Run Help Python Last edit was 2 minutes ago

Run all Serverless Schedule Comments Share

Just now (2s) 26

```
%sql
SELECT restaurant_id, count(*) AS total_orders
FROM orders_bronze
GROUP BY restaurant_id
ORDER BY total_orders DESC
LIMIT 10;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = (restaurant_id: string, total_orders: long)

Table	restaurant_id	total_orders
1	R005	114
2	R198	111
3	R031	110
4	R083	108
5	R171	108
6	R131	108
7	R058	107
8	R092	106
9	R138	104
10	R151	104

10 rows 1.960s runtime Refreshed Now

12. Item-Level Insights (explode items_ordered)

- Find the most frequently ordered items

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

analysis

Python Last edit was 2 minutes ago

Run all Serverless Schedule Comments Share

2 minutes ago (2s) 28

```
%sql
SELECT item_id, item_name, count(*) as number_of_times_ordered
FROM orders_bronze
LATERAL VIEW EXPLODE(items_ordered) exploded AS item
GROUP BY item_id, item_name
ORDER BY number_of_times_ordered DESC
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = (item_id: string, item_name: string ... 1 more field)

Table	item_id	item_name	number_of_times_ordered
1	I0357	Argue Cake	111
2	I0011	Purpose Brownie	104
3	I0274	Even Tea	103
4	I0074	Painting Coffee	101
5	I0145	Rich Biryani	101
6	I0254	Sell Ice Cream	101
7	I0255	Finish Garlic Bread	100
8	I0090	Others Wrap	100
9	I0189	Market Spring Roll	99
10	I0087	Student Tea	99
11	I0093	Street Pizza	98
12	I0492	Bring Cake	98
13	I0371	Thought Pizza	98
14	I0015	Newspaper Tea	97
15	I0375	Very Soda	97

500 rows 2.042s runtime Refreshed 2m ago

- Find the average number of items per order

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

analysis

Python Last edit was now

Run all Serverless Schedule Comments Share

Just now (2s) 29

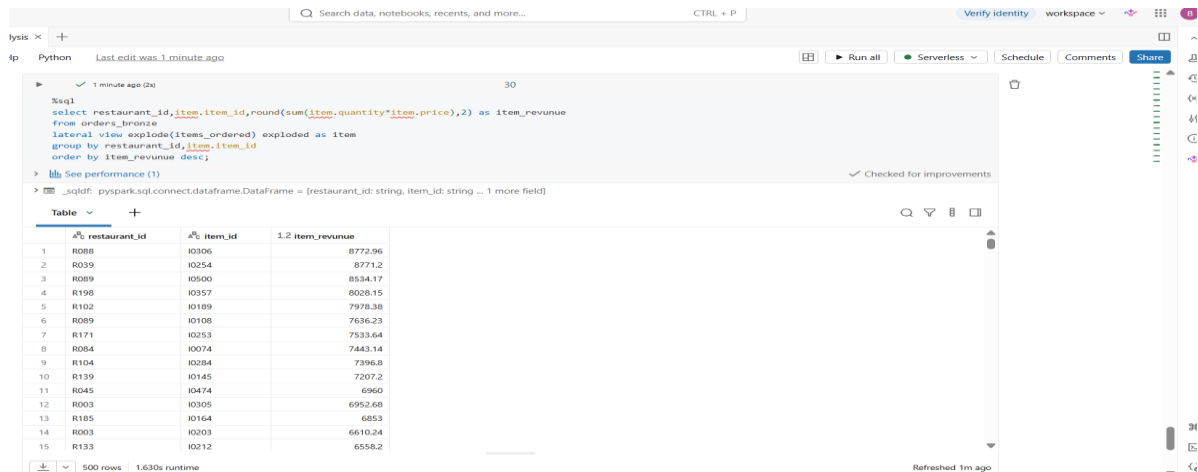
```
%sql
select round(avg(size(items_ordered)),2) as average_items_per_order
from orders_bronze;
```

See performance (1) ✓ Checked for improvements

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [average_items_per_order: double]

Table	average_items_per_order
1	1.65

- Find item revenue contribution per restaurant

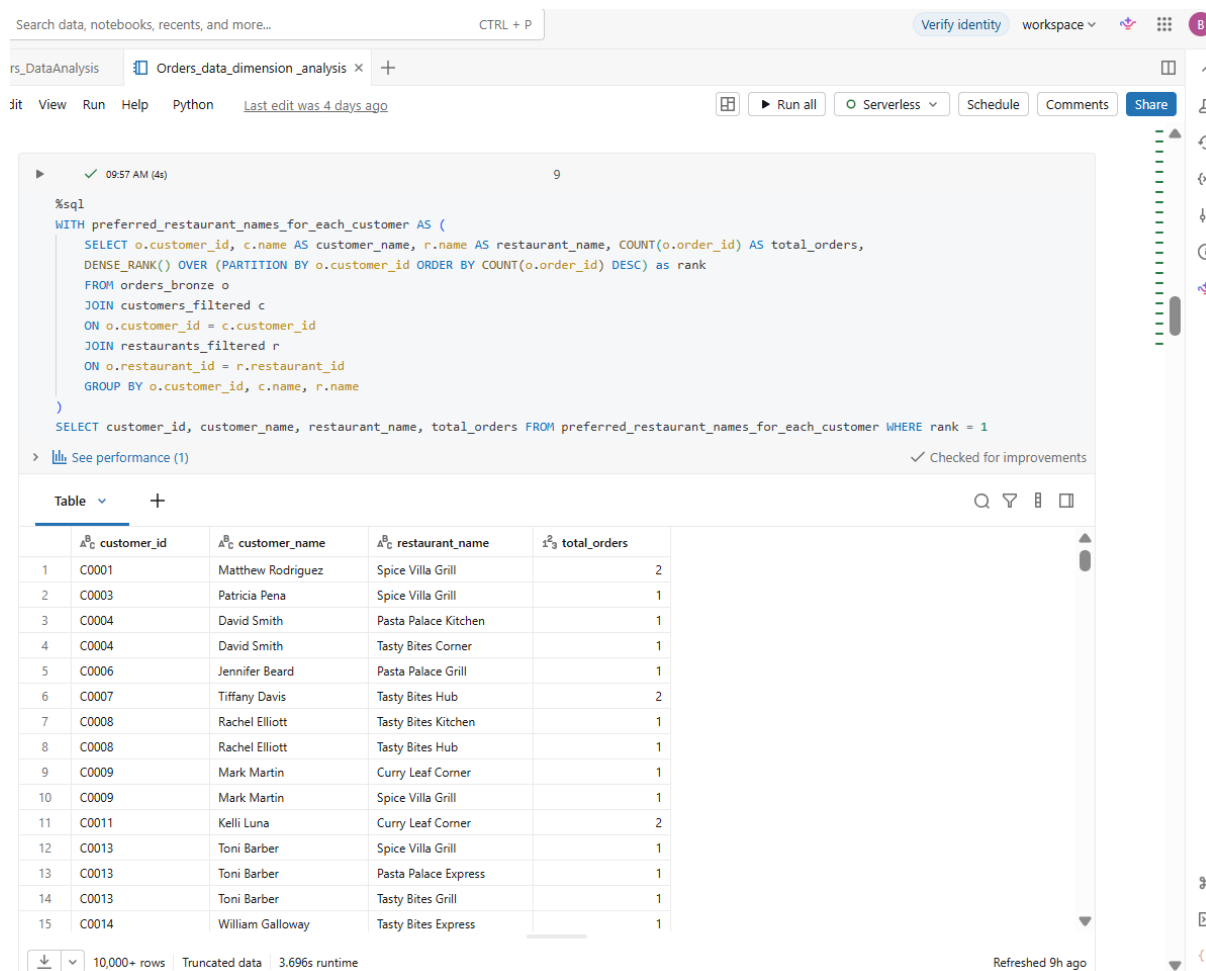


```
%sql
select restaurant_id, item_id, round(sum(item.quantity*item.price),2) as item_revenue
from orders_bronze
lateral view explode(items_ordered) exploded as item
group by restaurant_id, item_id
order by item_revenue desc;
```

	restaurant_id	item_id	item_revenue
1	R008	I0306	8772.96
2	R039	I0254	8771.2
3	R009	I0500	8534.17
4	R198	I0357	8028.15
5	R102	I0189	7978.38
6	R089	I0108	7636.23
7	R171	I0253	7533.64
8	R084	I0074	7443.14
9	R104	I0284	7396.8
10	R139	I0145	7207.2
11	R045	I0474	6960
12	R003	I0305	6952.68
13	R185	I0164	6853
14	R003	I0203	6610.24
15	R133	I0212	6558.2

Orders Data and Dimension Datasets Analysis

1. Find the preferred restaurant names for each customer



```
%sql
WITH preferred_restaurant_names_for_each_customer AS (
  SELECT o.customer_id, c.name AS customer_name, r.name AS restaurant_name, COUNT(o.order_id) AS total_orders,
  DENSE_RANK() OVER (PARTITION BY o.customer_id ORDER BY COUNT(o.order_id) DESC) as rank
  FROM orders_bronze o
  JOIN customers_filtered c
  ON o.customer_id = c.customer_id
  JOIN restaurants_filtered r
  ON o.restaurant_id = r.restaurant_id
  GROUP BY o.customer_id, c.name, r.name
)
SELECT customer_id, customer_name, restaurant_name, total_orders FROM preferred_restaurant_names_for_each_customer WHERE rank = 1
```

	customer_id	customer_name	restaurant_name	total_orders
1	C0001	Matthew Rodriguez	Spice Villa Grill	2
2	C0003	Patricia Pena	Spice Villa Grill	1
3	C0004	David Smith	Pasta Palace Kitchen	1
4	C0004	David Smith	Tasty Bites Corner	1
5	C0006	Jennifer Beard	Pasta Palace Grill	1
6	C0007	Tiffany Davis	Tasty Bites Hub	2
7	C0008	Rachel Elliott	Tasty Bites Kitchen	1
8	C0008	Rachel Elliott	Tasty Bites Hub	1
9	C0009	Mark Martin	Curry Leaf Corner	1
10	C0009	Mark Martin	Spice Villa Grill	1
11	C0011	Kelli Luna	Curry Leaf Corner	2
12	C0013	Toni Barber	Spice Villa Grill	1
13	C0013	Toni Barber	Pasta Palace Express	1
14	C0013	Toni Barber	Tasty Bites Grill	1
15	C0014	William Galloway	Tasty Bites Express	1

2. Find the top 10 areas receiving orders in each month

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis × +

dit View Run Help Python Last edit was 4 days ago

Run all Serverless Schedule Comments Share

```
%sql
WITH areas_receiving_orders_per_month AS(
  SELECT date_format(o.timestamp, 'MMMM') as month, l.area AS area, COUNT(o.order_id) AS total_orders,
         DENSE_RANK() OVER (PARTITION BY date_format(o.timestamp, 'MMMM') ORDER BY COUNT(o.order_id) DESC) AS rank
  FROM orders_bronze o
  JOIN locations_filtered l
  ON o.delivery_location_id = l.location_id
  GROUP BY date_format(o.timestamp, 'MMMM'), l.area
)
SELECT month, area, total_orders FROM areas_receiving_orders_per_month WHERE rank <= 10 ORDER BY month, rank;
```

See performance (1) ✓ Checked for improvements

	month	area	total_orders
1	February	Elizabeth Locks	84
2	February	John Underpass	81
3	February	Rodriguez Mill	79
4	February	Hill Station	76
5	February	Padilla Mall	75
6	February	Smith Garden	75
7	February	Reed Glens	74
8	February	Harper Street	72
9	February	Patrick Mission	71
10	February	Joshua Drive	71
11	February	Flores Vista	70
12	February	Alex Curve	69
13	January	Rodriguez Mill	87
14	January	Eric Hills	86
15	January	Hill Station	80

40 rows 2.709s runtime Refreshed 9h ago

3. Find all successfully delivered orders that were paid using a card, to analyze payment preferences and delivery success rates

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis × +

Edit View Run Help Python Last edit was 4 days ago

Run all Serverless Schedule Comments Share

```
%sql
SELECT o.*, c.name AS customer_name
FROM orders_bronze o
JOIN customers_filtered c ON o.customer_id = c.customer_id
WHERE o.order_status = 'delivered' AND o.payment_method = 'card';
```

See performance (1) ✓ Checked for improvements

	agent_id	customer_id	delivery_location_id	items_ordered
1	A086	C5605	L089	[{"item_id": "I0491", "item_name": "Talk Pasta", "price": 10.6, "quantity": 2}]
2	A352	C7525	L082	[{"item_id": "I0403", "item_name": "Outside Juice", "price": 23.61, "quantity": 1}, {"item_id": "I0018", "item_name": "Outside Juice", "price": 23.61, "quantity": 1}]
3	A068	C1909	L083	[{"item_id": "I0038", "item_name": "Near Soda", "price": 15.97, "quantity": 4}, {"item_id": "I0134", "item_name": "Near Soda", "price": 15.97, "quantity": 4}]
4	A196	C3246	L022	[{"item_id": "I0185", "item_name": "Today Nachos", "price": 22.46, "quantity": 5}]
5	A202	C8344	L100	[{"item_id": "I0160", "item_name": "Almost Brownie", "price": 13.71, "quantity": 4}, {"item_id": "I0187", "item_name": "Almost Brownie", "price": 13.71, "quantity": 4}]
6	A390	C3024	L051	[{"item_id": "I0254", "item_name": "Sell Ice Cream", "price": 27.41, "quantity": 2}]
7	A072	C2294	L082	[{"item_id": "I0388", "item_name": "Nor Brownie", "price": 9.17, "quantity": 2}]
8	A307	C8811	L024	[{"item_id": "I0112", "item_name": "Run Pizza", "price": 25.36, "quantity": 1}, {"item_id": "I0049", "item_name": "Run Pizza", "price": 25.36, "quantity": 1}]
9	A279	C1179	L038	[{"item_id": "I0470", "item_name": "Radio Brownie", "price": 25.75, "quantity": 2}]
10	A157	C2504	L062	[{"item_id": "I0066", "item_name": "Reason Cake", "price": 27.3, "quantity": 1}, {"item_id": "I0158", "item_name": "Reason Cake", "price": 27.3, "quantity": 1}]
11	A045	C5302	L100	[{"item_id": "I0285", "item_name": "Community Garlic Bread", "price": 7.49, "quantity": 5}]
12	A123	C0254	L024	[{"item_id": "I0003", "item_name": "Road Nachos", "price": 9.5, "quantity": 1}]
13	A184	C3267	L091	[{"item_id": "I0269", "item_name": "Technology Biryani", "price": 7.24, "quantity": 3}, {"item_id": "I0155", "item_name": "Technology Biryani", "price": 7.24, "quantity": 3}]
14	A103	C0249	L025	[{"item_id": "I0436", "item_name": "Choose Ice Cream", "price": 0, "quantity": 1}]
15				

1,010 rows 2.092s runtime Refreshed 9h ago

4. Which cuisines generate the highest average order value

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Orders_data_dimension_analysis

Edit View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

```
%sql
SELECT r.cuisines, ROUND(AVG(o.total_amount), 2) AS avg_total_amount
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
GROUP BY r.cuisines ORDER BY avg_total_amount DESC
```

See performance (1) Checked for improvements

	cuisines	avg_total_amount
1	Thai,Chinese	160.1
2	Thai,Italian,Chinese	156.68
3	Mexican,American	146.19
4	Mexican,Indian,Thai	144.91
5	Mexican,American,Thai	143.95
6	American,Mexican	123.96
7	Italian,Indian,Mexican	121.05
8	American,Chinese	118.95
9	American,Italian	110.58
10	Chinese,Mexican	109.39
11	Mexican,American,Italian	108.58
12	Mexican,Italian,Chinese	105.89
13	American,Thai,Chinese	105.23
14	Thai,Indian	103.06
15	Indian,American,Chinese	102.62

67 rows 2.260s runtime Refreshed 9h ago

5. Which delivery agents have delivered the most high-value orders (above \$50)

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

Orders_data_dimension_analysis

Edit View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

```
%sql
SELECT d.agent_id AS delivery_agent,d.name AS delivery_agent_name, COUNT(o.order_id) as total_orders
FROM orders_bronze o
JOIN delivery_agents_filtered d ON o.agent_id = d.agent_id
WHERE o.total_amount > 50
GROUP BY d.agent_id, d.name
ORDER BY total_orders DESC
```

See performance (1) Checked for improvements

	delivery_agent	delivery_agent_name	total_orders
1	A309	Christopher Chase	213
2	A207	Christopher Williams	171
3	A393	Justin Lam	118
4	A350	Charlene Morgan	112
5	A370	Joan Harrison	110
6	A138	Carrie Maxwell	108
7	A178	John King	108
8	A360	Justin Smith	106
9	A168	James Osborn	104
10	A245	Teresa Ryan	102
11	A056	Robert Gonzales	102
12	A212	David King	96
13	A230	Scott Fields	96
14	A315	Scott Gonzalez	94
15	A174	Stephen Ramsey	94

350 rows 2.286s runtime Refreshed 9h ago

6. Which cities (area_name) are generating the most revenue for Italian restaurants

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis ×

Run all Serverless Schedule Comments Share

09:57 AM (3s) 19

```
%sql
SELECT l.area AS area_name, ROUND(SUM(o.total_amount), 2) AS total_revenue_dollars
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
JOIN locations_filtered l ON o.delivery_location_id = l.location_id
WHERE r.cuisines LIKE '%Italian%'
GROUP BY l.area
ORDER BY total_revenue_dollars DESC
```

See performance (1) ✓ Checked for improvements

	area_name	total_revenue_dollars
1	John Underpass	4077.09
2	Brown Spring	3811.75
3	Donald Row	3759.01
4	Jimmy Center	3659.54
5	Kirk Valley	3655.7
6	Padilla Mall	3601.73
7	Dorothy Forest	3598.22
8	Eric Hills	3587.16
9	Karen Cliff	3499.69
10	Gomez Hollow	3393.3
11	Smith Estates	3367.53
12	Reed Glens	3356.24
13	Smith Rapids	3345.51
14	Smith Garden	3344.55
15	Hill Station	3342.75

100 rows 2.584s runtime Refreshed 9h ago

7. What is the tip-to-total ratio by cuisine type

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis ×

Run all Serverless Schedule Comments Share

09:57 AM (3s) 21

```
%sql
SELECT r.cuisines AS cuisines, ROUND(AVG(o.tip/o.total_amount), 2) AS tip_to_total_ratio
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
WHERE o.total_amount > 0
GROUP BY r.cuisines
ORDER BY tip_to_total_ratio DESC
```

See performance (1) ✓ Checked for improvements

	cuisines	tip_to_total_ratio
1	Indian,Mexican,American	0.22
2	Thai,American,Indian	0.17
3	Chinese,Thai	0.16
4	Chinese,American	0.1
5	Chinese,Italian,Thai	0.09
6	Italian,Thai,Chinese	0.09
7	Mexican,Italian,American	0.08
8	Chinese,Indian,American	0.08
9	Indian,American	0.08
10	Indian,American,Mexican	0.08
11	Chinese	0.07
12	Thai,Mexican,Italian	0.07
13	Mexican	0.07
14	Indian,Chinese,Mexican	0.07
15	American,Italian,Indian	0.07

67 rows 2.675s runtime Refreshed 9h ago

8. Which menu items are most frequently ordered, and from which restaurants

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis × +

dit View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

```
%sql
WITH exploded_orders AS (
  SELECT o.order_id, item.item_id
  FROM orders_bronze o
  LATERAL VIEW EXPLODE(o.items_ordered) AS item
)
SELECT m.item_name, m.restaurant_id, COUNT(*) AS times_ordered
FROM exploded_orders eo
JOIN menu_items_filtered m ON eo.item_id = m.item_id
GROUP BY m.item_name, m.restaurant_id
ORDER BY times_ordered DESC;
```

See performance (1) ✓ Checked for improvements

	Item_name	restaurant_id	times_ordered
1	Argue Cake	R198	111
2	Purpose Brownie	R138	104
3	Even Tea	R090	103
4	Painting Coffee	R084	101
5	Sell Ice Cream	R039	101
6	Rich Biryani	R139	101
7	Finish Garlic Bread	R082	100
8	Others Wrap	R165	100
9	Market Spring Roll	R102	99
10	Student Tea	R158	99
11	Bring Cake	R100	98
12	Street Pizza	R011	98
13	Thought Pizza	R005	98
14	Very Soda	R131	97
15	Newspaper Tea	R141	97

500 rows | 2.383s runtime Refreshed 9h ago

9. Which agents have delivered the most diverse cuisines

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis × +

Edit View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

```
%sql
SELECT o.agent_id, COUNT(DISTINCT r.cuisines) AS distinct_cuisines
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
GROUP BY o.agent_id
ORDER BY COUNT(DISTINCT r.cuisines) DESC;
```

See performance (1) ✓ Checked for improvements

	agent_id	distinct_cuisines
1	A309	43
2	A360	40
3	A207	39
4	A393	39
5	A168	38
6	A138	37
7	A245	37
8	A162	36
9	A325	36
10	A212	35
11	A356	34
12	A350	34
13	A381	34
14	A230	34
15	A258	34

400 rows | 1.991s runtime Refreshed 9h ago

10. What is the average customer lifetime value by signup year

Q Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

DataAnalysis Orders_data_dimension_analysis × +

lit View Run Help Python Last edit was 4 days ago

Run all Serverless Schedule Comments Share

```
%sql
WITH customer_life AS (
    SELECT
        YEAR(c.signup_date) AS signup_year, c.customer_id, SUM(o.total_amount) AS total_amount_spent_per_customer
    FROM orders_bronze o
    JOIN customers_filtered c ON o.customer_id = c.customer_id
    GROUP BY YEAR(c.signup_date), c.customer_id
)
SELECT
    signup_year,
    ROUND(AVG(total_amount_spent_per_customer), 2) AS avg_ltv_dollars
FROM customer_life
GROUP BY signup_year
ORDER BY signup_year;
```

> See performance (1)

✓ Checked for improvements

	1.1 signup_year	1.2 avg_ltv_dollars
1	1954	22.64
2	1955	172.65
3	1956	139.9
4	1957	155.41
5	1958	135.53
6	1959	141.08
7	1960	173.02
8	1961	222.25
9	1962	192.06
10	1963	137.26
11	1964	195.17
12	1965	163.28
13	1966	136.78
14	1967	168.5
15	1968	137.79

72 rows 1.877s runtime Refreshed 9h ago

11. Which restaurants have the highest order frequency per location

rs_DataAnalysis Orders_data_dimension_analysis × +

dit View Run Help Python Last edit was 4 days ago

Run all Serverless Schedule Comments Share

```
%sql
SELECT r.restaurant_id, l.location_id, COUNT(o.order_id) AS total_orders
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
JOIN locations_filtered l ON r.location_id = l.location_id
GROUP BY r.restaurant_id, l.location_id
ORDER BY total_orders DESC
```

> See performance (1)

	restaurant_id	location_id	total_orders
1	R005	L039	114
2	R198	L022	111
3	R171	L093	108
4	R083	L069	108
5	R092	L057	106
6	R185	L095	104
7	R035	L022	104
8	R090	L046	103
9	R146	L047	103
10	R180	L086	103
11	R094	L047	103
12	R134	L033	102
13	R117	L069	102
14	R014	L039	102
15	R121	L091	101

137 rows 2.696s runtime Refreshed 9h ago

12. Which locations are prone to order cancellation

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

rs_DataAnalysis Orders_data_dimension_analysis × +

lit View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

09:57 AM (2s) 31

```
%sql
SELECT l.area AS area_name, COUNT(*) AS total_cancelled_orders
FROM orders_bronze o
JOIN locations_filtered l ON o.delivery_location_id = l.location_id
WHERE o.order_status = 'cancelled'
GROUP BY l.area
ORDER BY total_cancelled_orders DESC
```

See performance (1) ✓ Checked for improvements

	area_name	total_cancelled_orders
1	Evans Stream	58
2	Elizabeth Locks	57
3	Flores Vista	55
4	Ashley Extensions	53
5	Padilla Mall	52
6	Hill Station	52
7	Joshua Drive	51
8	Rodriguez Mill	51
9	White Tunnel	51
10	Curtis Oval	51
11	Arnold Fort	50
12	John Underpass	49
13	Carpenter Views	49
14	Hunt Drives	49
15	Lance Radial	49

13. Find top 10 customers who have ordered from the widest variety of restaurants

nts, and more... CTRL + P Verify identity workspace

rders_DataAnalysis Orders_data_dimension_analysis × +

on Last edit was 4 days ago Run all Serverless Schedule Comments Share

09:57 AM (2s) 33

```
%sql
SELECT c.customer_id, COUNT(DISTINCT o.restaurant_id) AS distinct_restaurants
FROM orders_bronze o
JOIN customers_filtered c ON o.customer_id = c.customer_id
GROUP BY c.customer_id
ORDER BY distinct_restaurants DESC
LIMIT 10;
```

See performance (1) ✓ Checked for improvements

	customer_id	distinct_restaurants
1	C7381	9
2	C0362	8
3	C2231	8
4	C6444	8
5	C1449	8
6	C5088	7
7	C1423	7
8	C0313	7
9	C9870	7
10	C5365	7

↓ 10 rows 2.386s runtime Refreshed 9h ago

14. Identify restaurants with frequent repeated customers

notebooks, recents, and more... CTRL + P Verify identity workspace

Orders_DataAnalysis Orders_data_dimension_analysis × +

Edit View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

```
%sql
SELECT o.restaurant_id, o.customer_id, COUNT(*) AS number_of_times_visited
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
GROUP BY o.restaurant_id, o.customer_id
HAVING COUNT(*) > 1
ORDER BY number_of_times_visited DESC
```

See performance (1) ✓ Checked for improvements

	restaurant_id	customer_id	number_of_times_visited
1	R002	C5990	2
2	R108	C5302	2
3	R136	C9836	2
4	R019	C1469	2
5	R125	C3838	2
6	R161	C5231	2
7	R188	C0579	2
8	R012	C0232	2
9	R038	C8626	2
10	R062	C4746	2
11	R070	C6228	2
12	R196	C0113	2
13	R157	C7698	2
14	R121	C5441	2
15	R117	C3410	2

50 rows 1.977s runtime Refreshed 9h ago

15. Which payment methods are most preferred for high-value orders across cuisines

search data, notebooks, recents, and more... CTRL + P Verify identity workspace

_DataAnalysis Orders_data_dimension_analysis × +

View Run Help Python Last edit was 4 days ago Run all Serverless Schedule Comments Share

```
%sql
SELECT r.cuisines, o.payment_method, COUNT(*) AS total_orders
FROM orders_bronze o
JOIN restaurants_filtered r ON o.restaurant_id = r.restaurant_id
WHERE o.total_amount > 40
GROUP BY r.cuisines, o.payment_method
ORDER BY COUNT(*) DESC
```

See performance (1) ✓ Checked for improvements

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [cuisines: string, payment_method: string ... 1 more field]

	cuisines	payment_method	total_orders
1	Indian	card	220
2	Indian	wallet	214
3	Mexican	wallet	208
4	Indian	UPI	199
5	Mexican	cash	199
6	Indian	cash	195
7	Mexican	card	189
8	Mexican	UPI	186
9	American	card	125
10	American	UPI	122
11	American	cash	118
12	American	wallet	117
13	Thai	cash	115
14	Thai	card	113
15	Thai	wallet	103

260 rows 2.081s runtime Refreshed 9h ago

DLT Pipelines

DELTA LIVE TABLES

APRIL-MAY-JUNE DATASET INGESTED

Catalog

Serverless Starter Ware... Serverless 2XS

Catalog

Type to search

workspace

system

databricks_frst

dinedash_catalog

default

dinedash_schema

Tables (11)

customers_bronze

customers_filtered

delivery_agents_bronze

delivery_agents_filtered

locations_bronze

locations_filtered

menu_items_bronze

menu_items_filtered

orders_bronze

restaurants_bronze

restaurants_filtered

Volumes (1)

dinedash_volume

information_schema

Catalog Explorer > dinedash_catalog > dinedash_schema >

dinedash_volume

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume / dinedash_source / stream_data Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
orders_2024_01.json	2.46 MB	5 days ago
orders_2024_02.json	2.38 MB	5 days ago
orders_2024_03.json	2.09 MB	1 hour ago
orders_2024_04.json	3.34 MB	just now
orders_2024_05.json	2.31 MB	1 second ago
orders_2024_06.json	3.35 MB	just now

About this volume

Owner balusuramya19@gmail.com

Tags

Add tags

DLT PIPELINE SCRIPT CODE:

Step 1: Create Script

Bronze Layer Tables

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

Run pipeline

dl_t.pl

Pipeline x

Last edited 4 days ago

1

-- order bronze
CREATE OR REFRESH STREAMING TABLE orders_bronze AS
SELECT * FROM STREAM_READ_FILES("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/", format => "json");

2

-- customers bronze
CREATE LIVE TABLE customers_bronze AS
SELECT * FROM READ_FILES("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_customers.csv", format => "csv", header => true);

3

-- restaurants bronze
CREATE LIVE TABLE restaurants_bronze AS
SELECT * FROM READ_FILES("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_restaurants.csv", format => "csv", header => true);

4

-- locations bronze
CREATE LIVE TABLE locations_bronze AS
SELECT * FROM READ_FILES("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_locations.csv", format => "csv", header => true);

5

-- delivery agents bronze
CREATE LIVE TABLE agents_bronze AS
SELECT * FROM READ_FILES("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_delivery_agents.csv", format => "csv", header => true);

6

-- menu items bronze
CREATE LIVE TABLE items_bronze AS
SELECT * FROM READ_FILES("/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/dimensions/dim_menu_items.csv", format => "csv", header => true);

Silver Layer Tables

The screenshot displays a Databricks workspace interface. At the top, there's a search bar and a 'Verify identity' button. Below the search bar, the workspace name 'workspace' is visible. The main area shows a pipeline named 'dlt_pl' with a star icon. The pipeline is in a 'Run' state, with buttons for 'Connect', 'Dry run', and 'Run pipeline'. The pipeline's last edit was 4 days ago. The SQL code in the pipeline is as follows:

```
-- cleaned orders
CREATE OR REFRESH STREAMING TABLE cleaned_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_items EXPECT (size(items_ordered) > 0) ON VIOLATION DROP ROW
)
AS
SELECT
  order_id,
  timestamp AS order_ts,
  customer_id,
  restaurant_id,
  agent_id,
  delivery_location_id,
  items_ordered,
  explode(items_ordered) AS item,
  total_amount,
  tip,
  payment_method,
  order_status
FROM STREAM(LIVE.orders_bronze);
```

The interface also shows a sidebar with a search bar and a 'Verify identity' button. The pipeline is named 'LT_Pipeline' and is in a 'Run' state. The last edit was 4 days ago. The SQL code is the same as the one above.

Search data, notebooks, recents, and more...CTRL + PVerify identityworkspace

dtl.pl

edit was 4 days ago

8

SQL

```
-- orders_silver
CREATE OR REFRESH STREAMING TABLE orders_silver
(
  CONSTRAINT valid_restaurant_name EXPECT (restaurant_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_agent EXPECT (agent_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_item EXPECT (item_id IS NOT NULL) ON VIOLATION DROP ROW
)
AS
SELECT
  o.order_id,
  o.order_ts,
  o.customer_id,
  c.name AS customer_name,
  c.email,
  o.restaurant_id,
  r.name AS restaurant_name,
  r.cuisines,
  r.rating AS restaurant_rating,
  o.agent_id,
  a.name AS agent_name,
  a.rating AS agent_rating,
  o.delivery_location_id,
  l.city,
  o.item_id AS item_id,
  i.item_name,
  i.category,
  o.item_price AS item_price,
  o.item_quantity AS item_quantity,
  o.total_amount,
  o.tip,
  o.payment_method,
  o.order_status
FROM STREAM(LIVE.cleaned_orders) o
LEFT JOIN LIVE.customers_bronze c ON o.customer_id = c.customer_id
LEFT JOIN LIVE.restaurants_bronze r ON o.restaurant_id = r.restaurant_id
LEFT JOIN LIVE.agents_bronze a ON o.agent_id = a.agent_id
LEFT JOIN LIVE.items_bronze i ON o.item_id = i.item_id
LEFT JOIN LIVE.locations_bronze l ON o.delivery_location_id = l.location_id;
```

Search data, notebooks, recents, and more...CTRL + PVerify identityworkspace

dtl.pl

edit was 4 days ago

10

```
-- Orders with Items
CREATE OR REFRESH STREAMING TABLE orders_with_items AS
SELECT
  o.order_id,
  o.order_ts,
  o.restaurant_id,
  o.exploded_item_id AS item_id,
  i.item_name,
  i.category,
  o.exploded_price AS price,
  o.exploded_quantity AS quantity
FROM (
  SELECT
    order_id,
    timestamp AS order_ts,
    restaurant_id,
    explode(items_ordered) AS exploded
  FROM STREAM(LIVE.orders_bronze)
) o
LEFT JOIN LIVE.items_bronze i ON o.exploded_item_id = i.item_id;
```

Search data, notebooks, recents, and more...CTRL + PVerify identityworkspace

dtl.pl

edit was 4 days ago

11

```
-- orders with locations
CREATE OR REFRESH STREAMING TABLE orders_with_locations AS
SELECT
  o.order_id,
  o.order_ts,
  o.delivery_location_id,
  l.city,
  l.state,
  l.area
FROM STREAM(LIVE.cleaned_orders) o
LEFT JOIN LIVE.locations_bronze l ON o.delivery_location_id = l.location_id;
```

Search data, notebooks, recents, and more...CTRL + PVerify identityworkspace

dtl.pl

edit was 4 days ago

12

```
-- orders with customers
CREATE OR REFRESH STREAMING TABLE orders_with_customers AS
SELECT
  o.order_id,
  o.customer_id,
  c.name AS customer_name,
  o.total_amount,
  o.tip,
  o.payment_method,
  o.order_status
FROM STREAM(LIVE.cleaned_orders) o
LEFT JOIN LIVE.customers_bronze c ON o.customer_id = c.customer_id;
```

Gold Layer Tables

The screenshot shows a Databricks notebook with the following SQL queries:

```
-- Total orders per restaurant
CREATE LIVE TABLE total_orders_per_restaurant AS
SELECT restaurant_name, COUNT(DISTINCT order_id) AS total_orders
FROM LIVE.orders_silver
GROUP BY restaurant_name;
```

```
-- Top 10 selling items
CREATE LIVE TABLE top_selling_items AS
SELECT item_name, SUM(item_quantity) AS total_qty_sold
FROM LIVE.orders_silver
GROUP BY item_name
ORDER BY total_qty_sold DESC
LIMIT 10;
```

```
-- Avg delivery fees and tip by city
CREATE LIVE TABLE avg_fees_by_city AS
SELECT city, AVG(restaurant_rating) AS avg_restaurant_rating, AVG(tip) AS avg_tip
FROM LIVE.orders_silver
GROUP BY city;
```

```
-- Agent performance
CREATE LIVE TABLE agent_order_counts AS
SELECT agent_name, COUNT(DISTINCT order_id) AS orders_handled, AVG(agent_rating) AS avg_rating
FROM LIVE.orders_silver
GROUP BY agent_name
ORDER BY orders_handled DESC;
```

```
-- Revenue trend
CREATE LIVE TABLE daily_revenue_trend AS
SELECT DATE(order_ts) AS order_date, SUM(total_amount) AS total_revenue
FROM LIVE.orders_silver
GROUP BY order_date
ORDER BY order_date;
```

```
-- Customer lifetime value
CREATE LIVE TABLE customer_ltv AS
SELECT customer_id, customer_name, SUM(total_amount) AS lifetime_spend
FROM LIVE.orders_silver
GROUP BY customer_id, customer_name
ORDER BY lifetime_spend DESC;
```

```
-- Customer order counts
CREATE LIVE TABLE customer_order_counts AS
SELECT customer_name, COUNT(*) AS total_orders
FROM LIVE.orders_with_customers
GROUP BY customer_name;
```

Step 2: Create Pipeline

Configuration of the DineDash Delta Live Tables pipeline in Databricks

The screenshot shows the configuration page for a Databricks pipeline named `dlt_pl`. The pipeline is of type `ETL pipeline` and is triggered. The creator and owner are `balusuramya19@gmail.com`. The pipeline is configured to run as `balusuramya19@gmail.com`.

Code assets

- Root folder: `/Workspace/Users/bal...19@gmail.com/dlt_pl`
- Source code: `transformations/`

Default location for data assets

- Default catalog: `dinedash_catalog`
- Default schema: `pipeline`

Compute

- Serverless

Pipeline environment

- Environment version: `None`
- Beta

Configuration

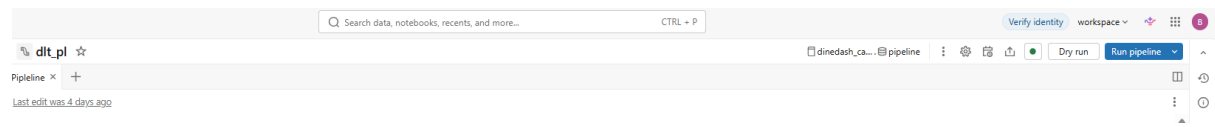
- Add configuration

Parameters

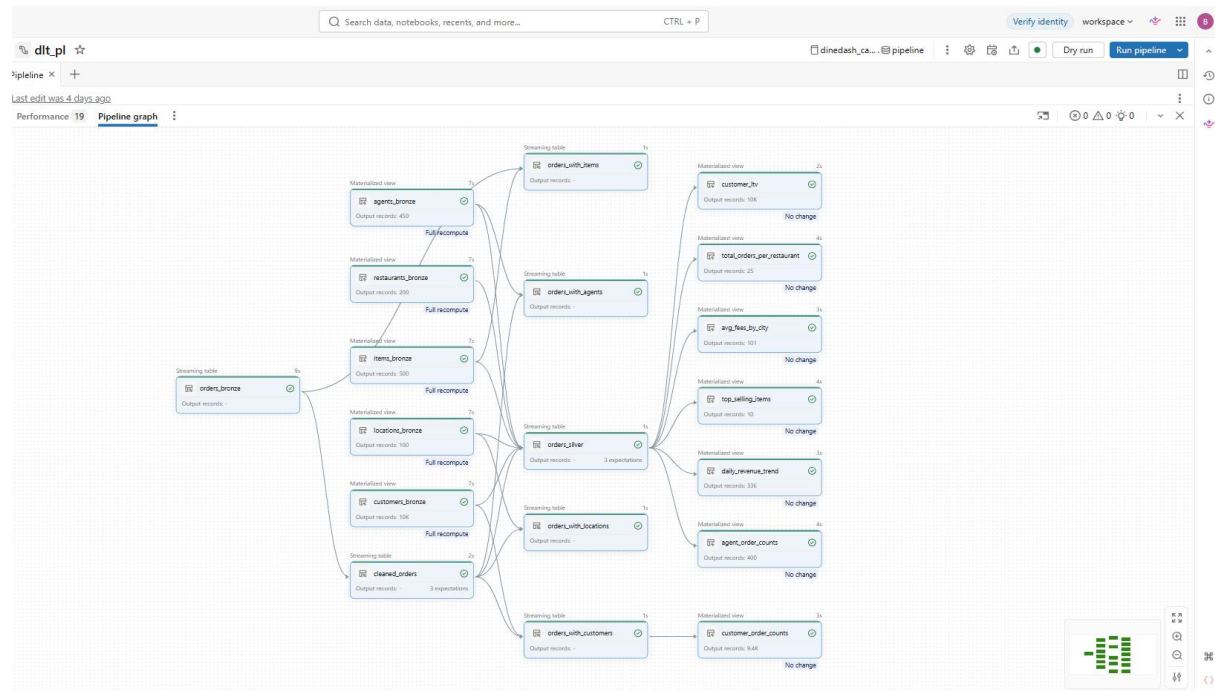
- Beta
- Add parameters

Step 3: Start the pipeline manually

Click on Run pipeline



PIPELINE EXECUTION (JANUARY-JUNE)



Observed table details from the pipeline

The screenshot shows the 'Run of DineDash_ETL_Pipeline' in the Databricks Jobs & Pipelines interface. The pipeline is in a 'Succeeded' state. The interface displays a graph of the pipeline execution, showing the flow of data between tables and views. The 'Table metrics' tab is selected, showing details for the 'orders_bronze' table, including its duration (6s), output records (38K), and expectations (N/A). The 'Details' panel on the right shows the pipeline's progress, with a green bar indicating 100% completion (18,311 records written, 0 records dropped).

Name	Duration	Output records	Expectations	Incrementalization
orders_bronze	6s	38K	-	N/A

Run of DineDash_ETL_Pipeline

dbc-cfc36f13-d2a5.cloud.databricks.com/jobs/262321225607139/runs/787167364744580?view=graph&o=7474653559930494

databricks

Jobs & Pipelines > DineDash Workflow > Run 834694923708483 > DineDash_ETL_Pipeline run Improved Performance Observability ON Succeeded

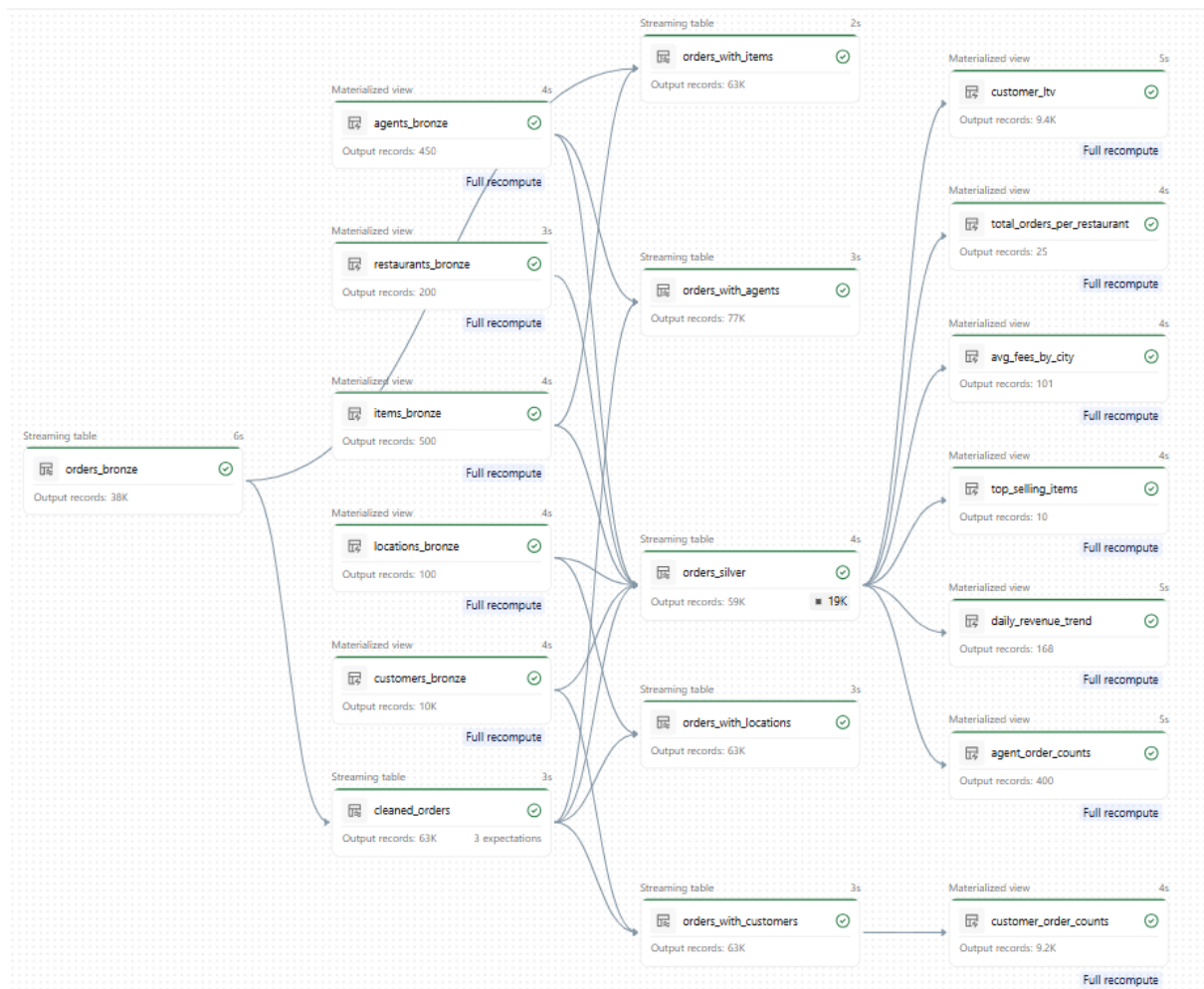
Graph Timeline List

Switch to graph dlt_gl

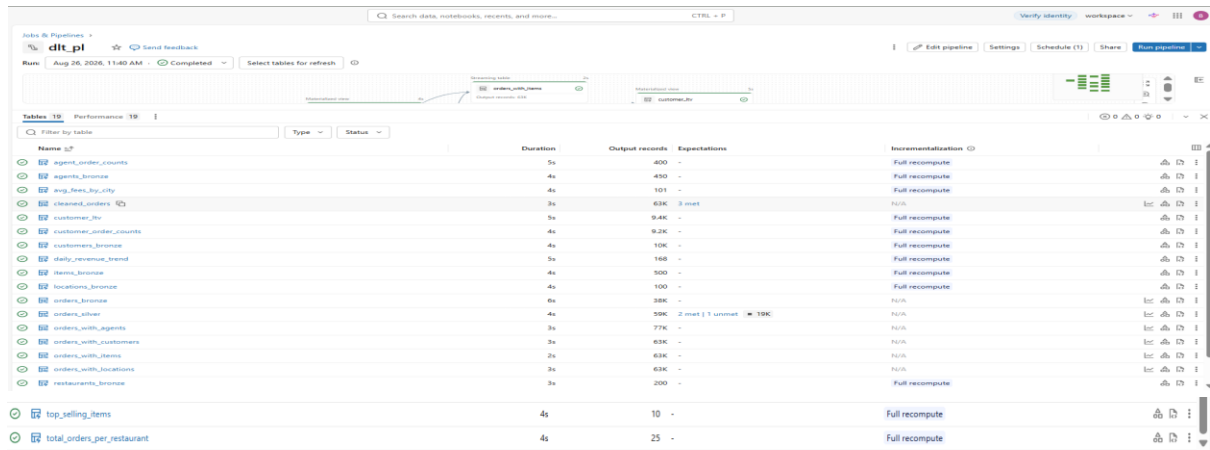
Streaming table: orders_bronze (6s, Output records: 38K) -> Materialized view: locations_bronze (4s, Output records: 100) -> Materialized view: customers_bronze (4s, Output records: 10K) -> Streaming table: cleaned_orders (3s, Output records: 63K, 3 expectations) -> Streaming table: orders_silver (4s, Output records: 59K, 19K) -> Streaming table: orders_with_locations (3s, Output records: 63K) -> Streaming table: orders_with_customers (3s, Output records: 63K) -> Materialized view: top_selling_items (4s, Output records: 10) -> Materialized view: daily_revenue_trend (5s, Output records: 168) -> Materialized view: agent_order_counts (5s, Output records: 400) -> Materialized view: customer_order_counts (4s, Output records: 9.2K)

Table metrics

Name	Duration	Output records	Expectations	Incrementalization
cleaned_orders	3s	63K	3 met	N/A



DLT pipeline execution showing processed tables, output records.



The screenshot shows the DLT pipeline execution interface. At the top, there's a search bar and a 'Verify identity' button. Below that, a 'Run' button is visible. The main section displays a list of tables with their performance metrics. The tables are listed in a table with columns: Name, Type, Status, Duration, Output records, Expectations, and Incrementalization. The tables are sorted by Name. The output records column shows the number of records processed for each table. The expectations column shows the number of expectations met. The incrementalization column shows the status of the incrementalization process.

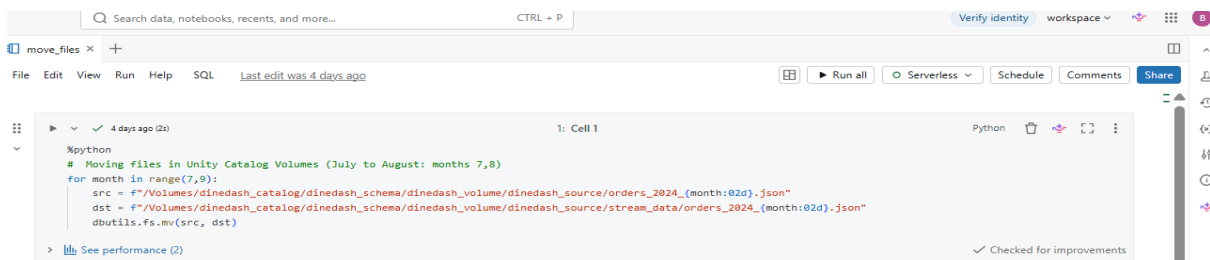
Name	Type	Status	Duration	Output records	Expectations	Incrementalization
agent_order_counts			5s	400	-	Full recompute
agents_bronze			4s	430	-	Full recompute
avg_items_by_city			4s	101	-	Full recompute
cleaned_orders			3s	63K	3 met	Full recompute
customer_tv			5s	9.4K	-	Full recompute
customer_order_counts			4s	6.2K	-	Full recompute
customer_bronze			4s	10K	-	Full recompute
daily_revenue_trend			5s	168	-	Full recompute
items_bronze			4s	500	-	Full recompute
locations_bronze			4s	100	-	Full recompute
orders_bronze			6s	38K	-	Full recompute
orders_silver			4s	59K	2 met 1 unmet	Full recompute
orders_with_agents			3s	77K	-	Full recompute
orders_with_customers			3s	63K	-	Full recompute
orders_with_items			2s	63K	-	Full recompute
orders_with_locations			2s	63K	-	Full recompute
restaurants_bronze			3s	200	-	Full recompute
top_selling_items			4s	10	-	Full recompute
total_orders_per_restaurant			4s	25	-	Full recompute

Step 4: Add New Files to Orders Folder

Now, add orders details for July and Aug month to the orders folder by executing below code in some notebook

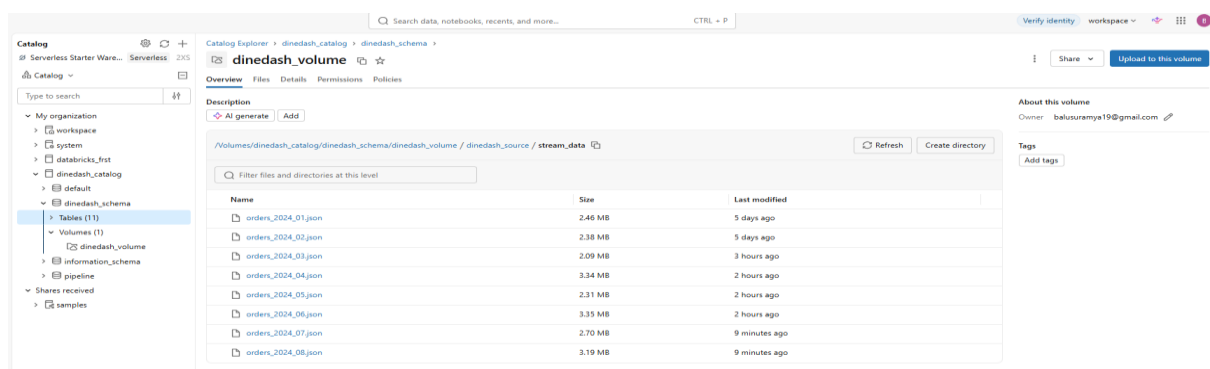
First, the **July and August 2024 order files** were manually uploaded to the `dinedash_source` folder. These files simulate the arrival of new monthly order data.

The files were then moved from the source folder to the **stream_data** folder using the following Python code



```
%python
# Moving files in Unity Catalog Volumes (July to August: months 7,8)
for month in range(7,9):
    src = f"/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/orders_2024_{month:02d}.json"
    dst = f"/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/orders_2024_{month:02d}.json"
    dbutils.fs.mv(src, dst)
```

July, August datasets Ingested



The screenshot shows the Unity Catalog Explorer interface. The left sidebar shows the catalog structure, including the 'dinedash_volume' and 'stream_data' folders. The main area displays a list of files in the 'stream_data' folder, showing the ingestion of July and August datasets. The files are listed with their names, sizes, and last modified dates.

Name	Size	Last modified
orders_2024_01.json	2.46 MB	5 days ago
orders_2024_02.json	2.38 MB	5 days ago
orders_2024_03.json	2.09 MB	3 hours ago
orders_2024_04.json	3.34 MB	2 hours ago
orders_2024_05.json	2.31 MB	2 hours ago
orders_2024_06.json	3.35 MB	2 hours ago
orders_2024_07.json	2.70 MB	9 minutes ago
orders_2024_08.json	3.19 MB	9 minutes ago

The screenshot displays a Databricks workspace with a Delta Lake pipeline. The pipeline is a sequence of tables, each represented as a Materialized view. The tables are: `restaurants_source` (Output records: 292), `items_source` (Output records: 130), `orders_source` (Output records: 124), `locations_source` (Output records: 100), `orders_driver` (Output records: 816), `customers_source` (Output records: 106), `cleaned_orders` (Output records: 86K), `total_order_per_restaurant` (Output records: 25), `avg_wait_time` (Output records: 101), `avg_rating_items` (Output records: 10), `avg_rating_restaurant` (Output records: 124), and `driver_per_order_count` (Output records: 692). The pipeline is triggered by a Streaming table. The status bar at the bottom shows 'cleaned_orders' with 36 tables, 86K output records, and 3 metrics.

Expectations	Flows
Written	100% (86,088)
Dropped	0% (0)

The screenshot displays the Databricks interface with a SQL query execution plan for a table named 'orders_pl'. The query is a complex join of multiple tables: restaurant_bronze, items_bronze, locations_bronze, customers_bronze, channels_bronze, orders_with_agents, orders_with_locations, and orders. The plan shows a sequence of joins and aggregations, with a final 'Full recursive' step. The bottom of the screen shows a table with columns: Name, Duration, Output records, Expectations, and Incremental. The table contains one row for 'orders_pl' with values: 4s, 81K, 2 met 11 unmet, and 26K.

Expectations	Flows
Written	75.6% (80,938)
Dropped	24.4% (26,181)

Expectations		All	Failures only
Name	Action	Fail%	Failed records
valid_restaurant_na...	DROP	24.4%	26181
valid_agent	DROP	0%	0
valid_item	DROP	0%	0

Step 6: Add remaining files onto orders folder(stream_data)

```
4 days ago (5s) 2

%python
# Moving files in Unity Catalog Volumes (July to August: months 9-12)
for month in range(9,13):
    src = f"/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/orders_2024_{month:02d}.json"
    dst = f"/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume/dinedash_source/stream_data/orders_2024_{month:02d}.json"
    dbutils.fs.mv(src, dst)

> See performance (4) ✓ Checked for improvements
```

Catalog Explorer > dineDash_catalog > dineDash_schema > dineDash_volume

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume / dineDash_source / stream_data

Filter files and directories at this level

Name	Size	Last modified
orders_2024_01.json	2.46 MB	6 days ago
orders_2024_02.json	2.38 MB	6 days ago
orders_2024_03.json	2.09 MB	4 hours ago
orders_2024_04.json	3.34 MB	3 hours ago
orders_2024_05.json	2.31 MB	3 hours ago
orders_2024_06.json	3.35 MB	3 hours ago
orders_2024_07.json	2.70 MB	58 minutes ago
orders_2024_08.json	3.19 MB	58 minutes ago
orders_2024_09.json	2.36 MB	17 minutes ago
orders_2024_10.json	3.23 MB	17 minutes ago
orders_2024_11.json	2.09 MB	17 minutes ago
orders_2024_12.json	2.17 MB	17 minutes ago

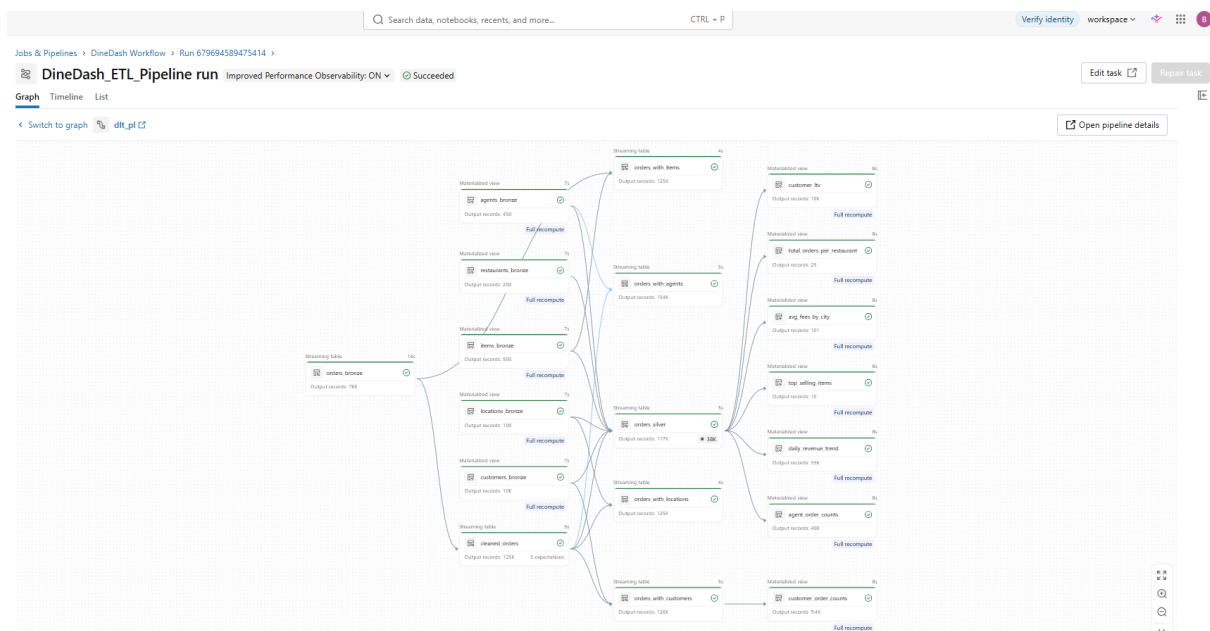
Refresh Create directory

My organization

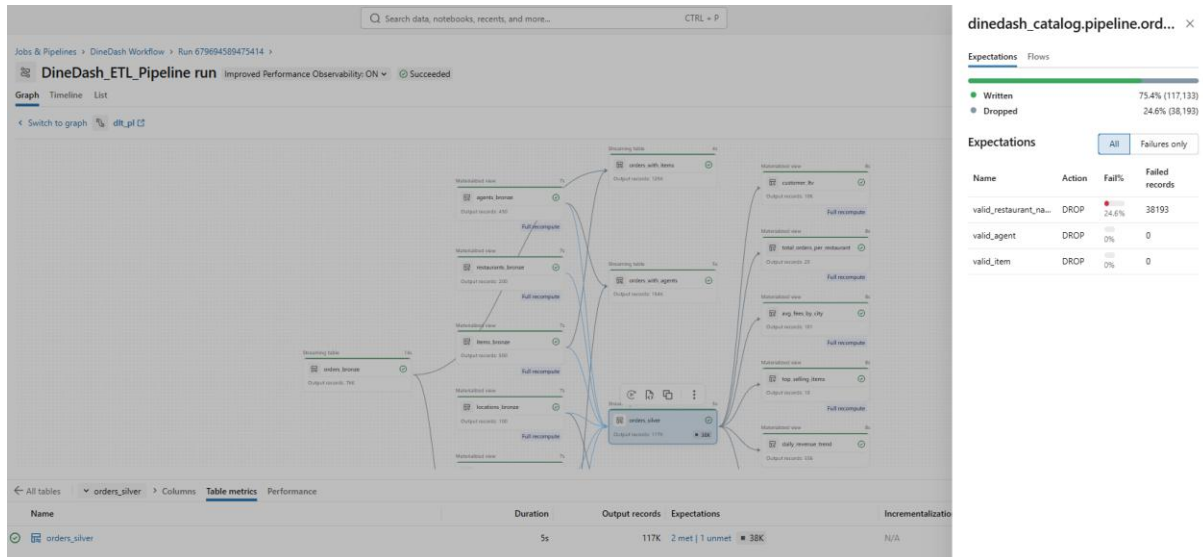
- workspace
- system
- dataBricks_fst
- dinedash_catalog
 - default
 - dinedash_schema
 - Tables (11)
 - customers_bronze
 - customers_filtered
 - delivery_agents_bronze
 - delivery_agents_filtered
 - locations_bronze
 - locations_filtered
 - menu_items_bronze
 - menu_items_filtered
 - orders_bronze
 - restaurants_bronze
 - restaurants_filtered
 - Volumes (1)
 - dinedash_volume
 - information_schema
 - pipeline

Step 7: Click on Start to execute the pipeline to feed data incrementally

Pipeline Execution (January-August)



Observed the tables after uploading September to December files



DLT pipeline execution showing processed tables, output records.

Jobs & Pipelines > **dlt.pl** ☆ Send feedback

Run: Aug 26, 2026, 01:32 PM - Completed - This update has already finished [Show the latest update](#)

Tables 19 Performance 19

Name	Duration	Output records	Expectations	Incrementalization
agent_order_counts	8s	400	-	Full recompute
agents_bronze	7s	450	-	Full recompute
avg_fees_by_city	8s	101	-	Full recompute
cleaned_orders	5s	125K	3 met	N/A
customer_tv	8s	10K	-	Full recompute
customer_order_counts	8s	9.4K	-	Full recompute
customers_bronze	7s	10K	-	Full recompute
daily_revenue_trend	8s	336	-	Full recompute
items_bronze	7s	500	-	Full recompute
locations_bronze	7s	100	-	Full recompute
orders_bronze	14s	76K	-	N/A
orders_silver	5s	117K	2 met 1 unmet 38K	N/A
orders_with_agents	5s	154K	-	N/A
orders_with_customers	5s	126K	-	N/A
orders_with_items	4s	125K	-	N/A
orders_with_locations	4s	125K	-	N/A
restaurants_bronze	7s	200	-	Full recompute
top_selling_items	8s	10	-	Full recompute
total_orders_per_restaurant	8s	25	-	Full recompute

WORKFLOWS

You will create a workflow to run the pipeline and publish some important insights onto Dashboard.

For this you create a Job in Databricks Workspace.

The Job you create will have following flow

Task 1: DLT Pipeline (which you have created earlier) execution

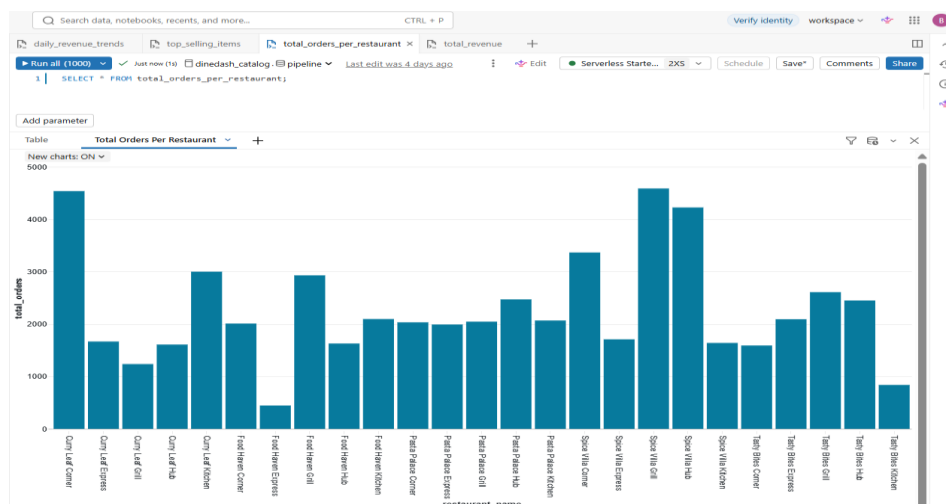
Task2: Creating Dashboard to display total_orders_per_restaurant, daily_revenue_trends, top_selling_items, total_revenue

Dashboards are fed from gold layer table query results and we add visualization on top of it

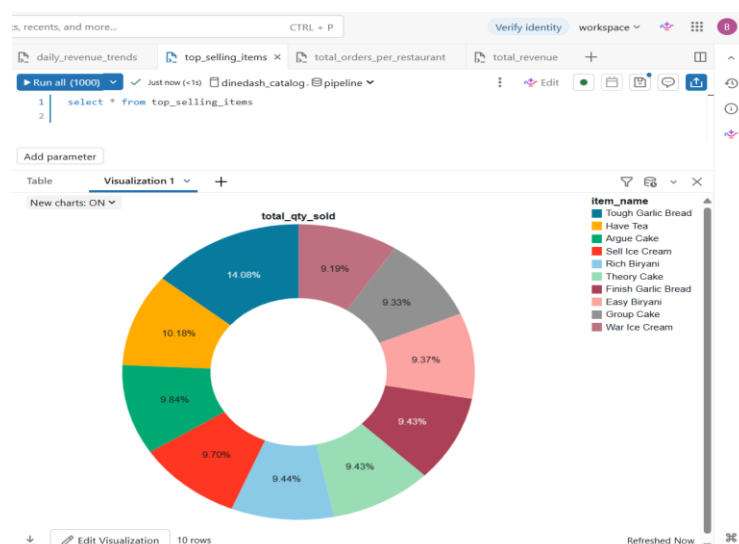
Dashboard

Create Visualizations

1. Total Orders per Restaurant



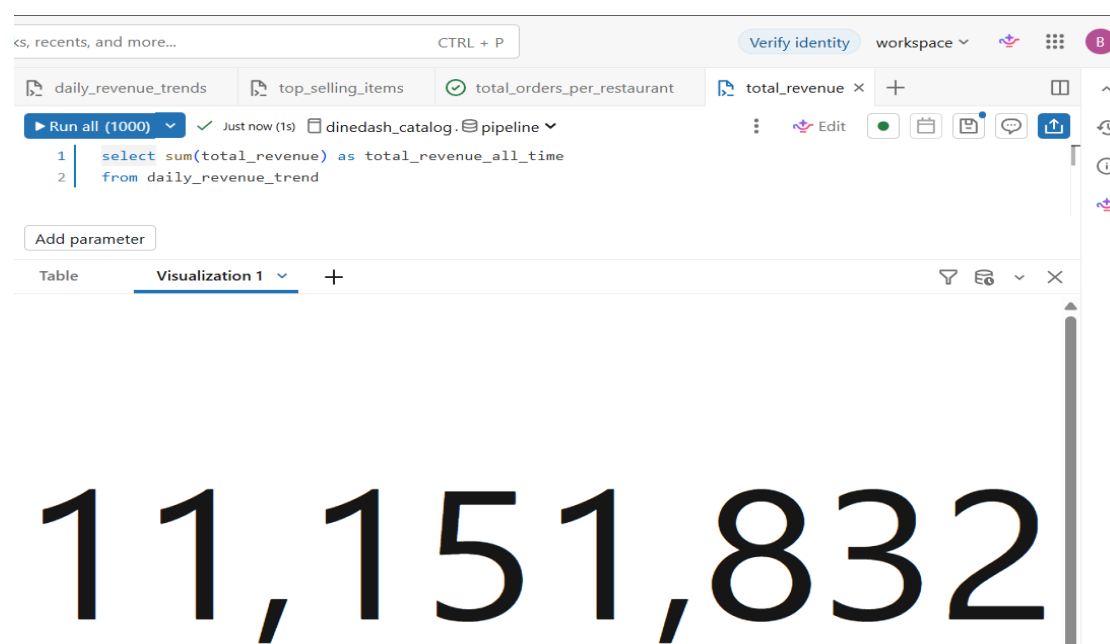
2. Top Selling Items



3. Daily Revenue Trends

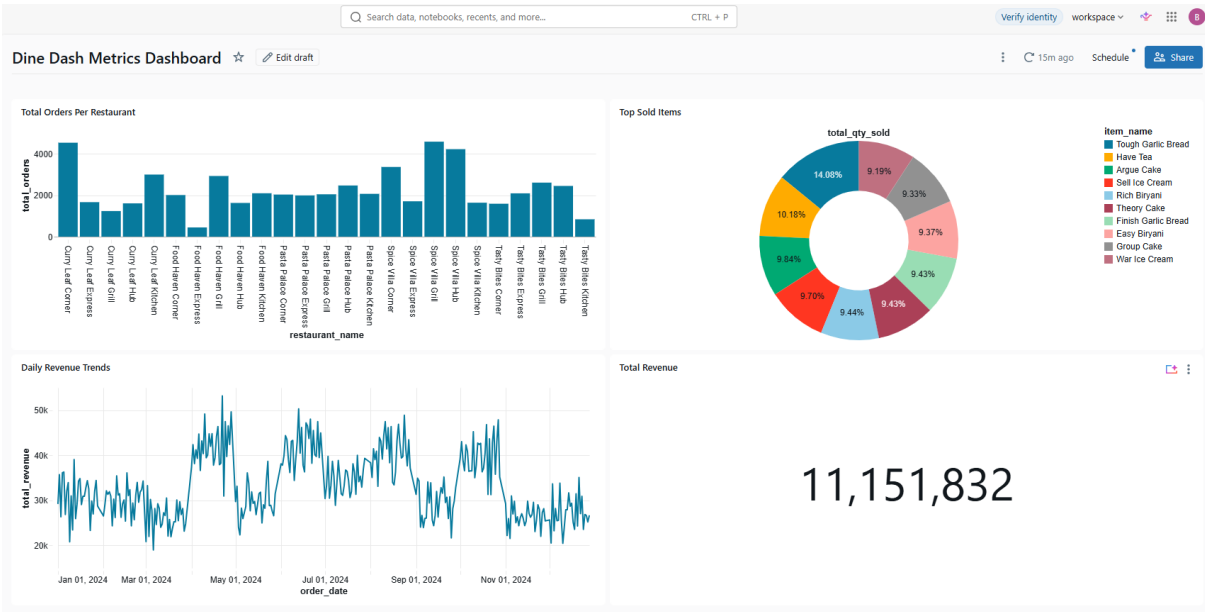


4. Total Revenue

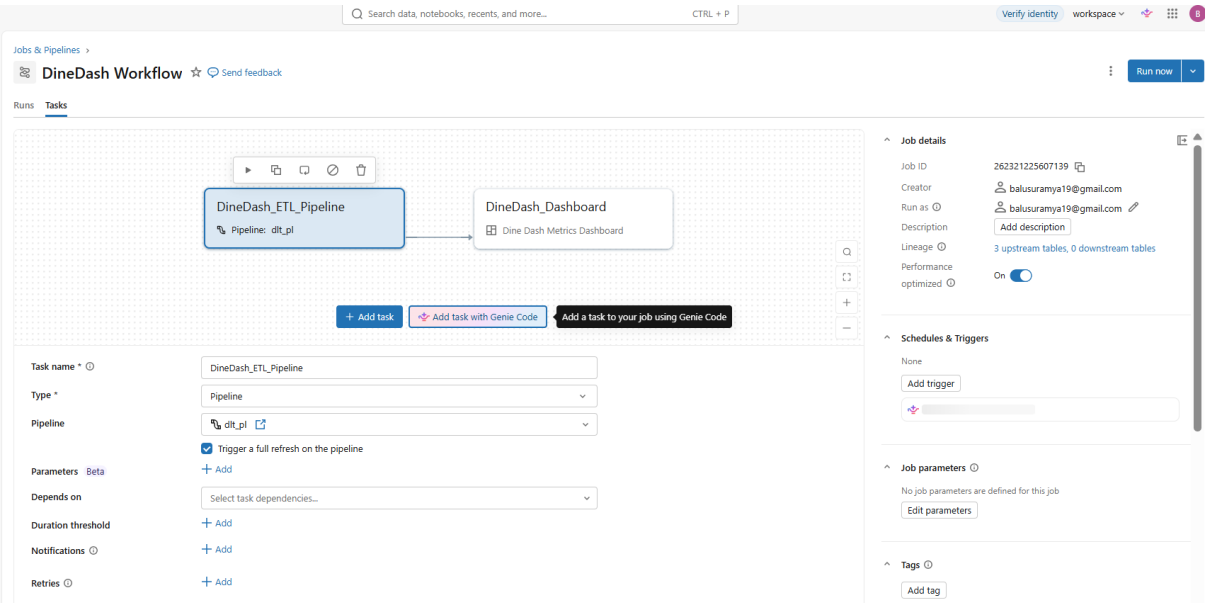


DASHBOARD (JANUARY-DECEMBER)

Once done, go to the dashboard menu in the workspace and publish the created dashboard with the default options



Creating Workflow/Job



Jobs & Pipelines > DineDash Workflow

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

Run now

Runs Tasks

DineDash_ETL_Pipeline Pipeline: dlt_pl

DineDash_Dashboard Dine Dash Metrics Dashboard

+ Add task + Add task with Genie Code

Task name * DineDash_Dashboard

Type * Dashboard

Dashboard * Dine Dash Metrics Dashboard

SQL warehouse * Serverless Starter Warehouse (2XS)

Subscribers * Select... Pause notifications

Dashboard filters * + Add

Depends on DineDash_ETL_Pipeline X

Run if dependencies * All succeeded

Duration threshold + Add

Modifiable + Add

Job details

Job ID 262321225607139

Creator balusuramya19@gmail.com

Run as balusuramya19@gmail.com

Description Add description

Lineage 3 upstream tables, 0 downstream tables

Performance optimized On

Schedules & Triggers

None

Add trigger

On updates to all of: dinedash_catalog.pipeline.daily_revenue_trend, dinedash_catalog.pipeline.top_selling_items, dinedash_catalog.pipeline.total_orders_per_restaurant

Reads three Unity Catalog tables; run when all are updated.

Apply Refine

Job parameters

No job parameters are defined for this job

Edit parameters

After first run, observe the results

Execute the Job by clicking on Run Now

Jobs & Pipelines > DineDash Workflow

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

DineDash Workflow run Improved Performance Observability: ON

Send feedback

Graph Timeline List

DineDash_ETL_Pipeline Succeeded - 2m 22s Pipeline: dlt_pl

DineDash_Dashboard Succeeded - 41s Dine Dash Metrics Dashboard

Job run details

Job ID 262321225607139

Job run ID 1100224176169869

Launched Manually

Started Aug 26, 2026, 02:31 PM

Ended Aug 26, 2026, 02:34 PM

Duration 3m 4s

Status Succeeded

Lineage 3 upstream tables, 0 downstream tables

Performance optimized Enabled

View run events

Now check with January to June month orderdatasets and run the job

Catalog Explorer > dinedash_catalog > dinedash_schema

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

dinedash_volume

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume / dinedash_source / stream_data

Refresh Create directory

Filter files and directories at this level

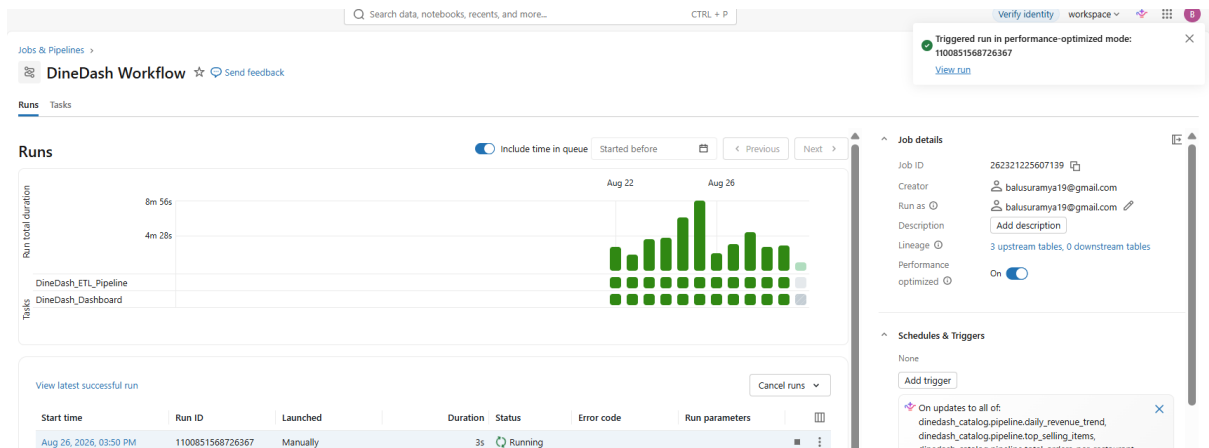
Name	Size	Last modified
orders_2024_01.json	2.46 MB	6 days ago
orders_2024_02.json	2.38 MB	6 days ago
orders_2024_03.json	2.09 MB	6 hours ago
orders_2024_04.json	3.34 MB	5 hours ago
orders_2024_05.json	2.31 MB	5 hours ago
orders_2024_06.json	3.35 MB	5 hours ago

About this volume

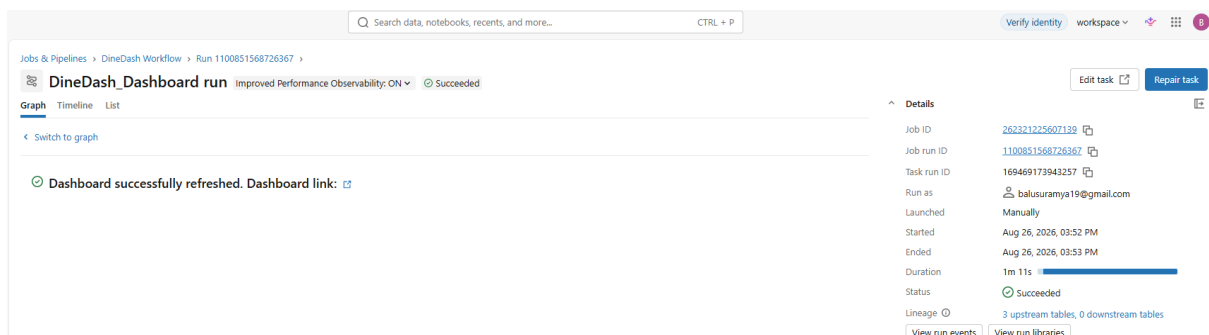
Owner balusuramya19@gmail.com

Tags

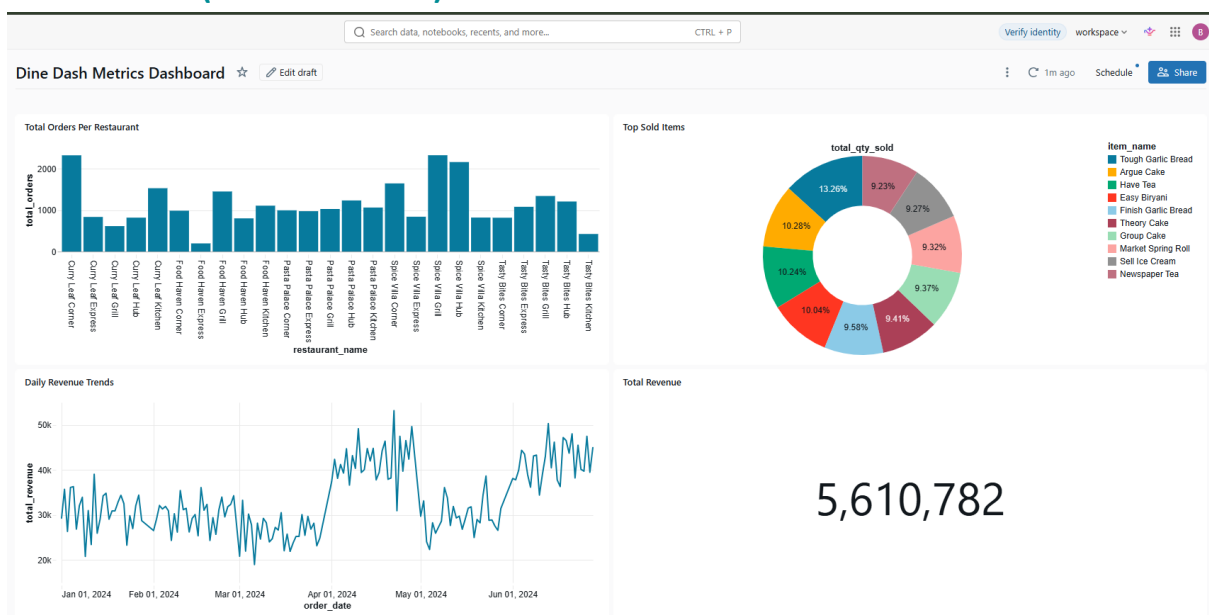
Add tags



Click on the dashboard task and that gives you a hyper link which takes you to the dashboard and then observe the metrics



DASHBOARD (JANUARY-JUNE)



Now add July to August month order datasets and run the job

Catalog

Serverless Starter Ware... Serverless 2XS

Catalog

Type to search

My organization

- workspace
- system
- databricks_first
- dinedash_catalog
 - default
 - dinedash_schema
 - Tables (11)
 - Volumes (1)
 - dinedash_volume
 - information_schema
 - pipeline
- Shares received
- samples

Catalog Explorer > dinedash_catalog > dinedash_schema >

dinedash_volume

Overview Files Details Permissions Policies

Description

AI generate Add

/Volumes/dinedash_catalog/dinedash_schema/dinedash_volume / dinedash_source / stream_data

Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
orders_2024_01.json	2.46 MB	6 days ago
orders_2024_02.json	2.38 MB	6 days ago
orders_2024_03.json	2.09 MB	7 hours ago
orders_2024_04.json	3.34 MB	5 hours ago
orders_2024_05.json	2.31 MB	5 hours ago
orders_2024_06.json	3.35 MB	5 hours ago
orders_2024_07.json	2.70 MB	1 second ago
orders_2024_08.json	3.19 MB	1 second ago

About this volume

Owner balusuramya19@gmail.com

Tags

Add tags

Jobs & Pipelines >

DineDash Workflow

Run now

Runs Tasks

Runs

Include time in queue Started before

< Previous Next >

Aug 22 Aug 26 Aug 26

Run total duration 8m 56s 4m 28s

Tasks

DineDash_ETL_Pipeline

DineDash_Dashboard

View latest successful run

Cancel runs

Start time	Run ID	Launched	Duration	Status	Error code	Run parameters
Aug 26, 2026, 04:00 PM	43157787122502	Manually	15s	Running		
Aug 26, 2026, 03:50 PM	1100851568726367	Manually	3m 15s	Succeeded		

Job details

Job ID 262321225607139

Creator balusuramya19@gmail.com

Run as balusuramya19@gmail.com

Description Add description

Lineage 3 upstream tables, 0 downstream tables

Performance optimized On

Schedules & Triggers

None

Add trigger

On updates to all of:
dinedash_catalog.pipeline.daily_revenue_trend,
dinedash_catalog.pipeline.top_selling_items,
dinedash_catalog.pipeline.total_orders_per_restaurant
Depends on three Unity Catalog tables; run when all are updated.

Observe the metrics

DASHBOARD (JANUARY-AUGUST)

Dine Dash Metrics Dashboard

1m ago Schedule Share

Total Orders Per Restaurant

4000

restaurant_name

Top Sold Items

total_qty_sold

Item_name

- Tough Garlic Bread
- Have Tea
- Theory Cake
- Argon Cake
- Easy Biryani
- Group Cake
- Recent Ice Cream
- Rich Biryani
- Student Tea
- Finish Garlic Bread

Daily Revenue Trends

50k

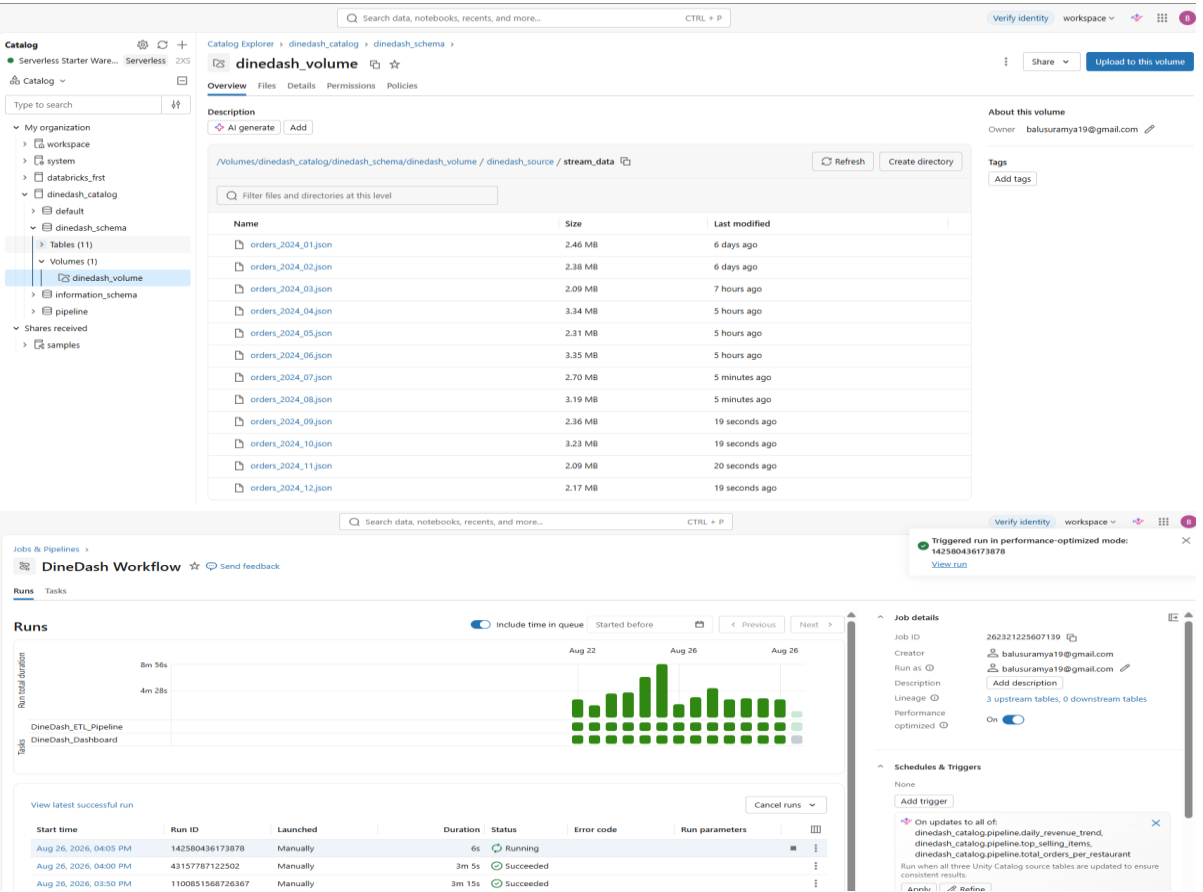
20k

order_date

Total Revenue

7,721,540

Now add the remaining datasets September to December month order files and run the job



Now repeat the same process and observe the metrics

FINAL PUBLISHED DASHBOARD

